
Efficient Training-Free Online Routing for High-Volume Multi-LLM Serving

Abstract

Increasing demand for Large Language Models (LLMs) services imposes substantial deployment and computation costs on providers. LLM routing offers a cost-efficient solution by directing queries to the optimal LLM based on model and query features. However, existing works primarily focus on offline scenarios and struggle to adapt to online settings with high query volume and constrained token budgets. In this work, we introduce the first training-free algorithm for online routing scenarios. Our algorithm leverages approximate nearest neighbor search to efficiently estimate query features and performs a one-time optimization over a small set of initial queries to learn a routing strategy that guides future routing. We provide theoretical guarantees demonstrating that our algorithm achieves a competitive ratio of $1 - o(1)$ under natural assumptions, which is further validated by extensive experiments across 3 benchmark datasets and 8 baselines, showing an average improvement of $3.55\times$ in overall performance, $1.85\times$ in cost efficiency, and nearly $4.25\times$ in throughput. Our code is available at

1 Introduction

The ability of Large Language Models (LLMs) to effectively interpret diverse domain knowledge has rapidly transformed the landscape of automated information processing [17, 34, 64]. However, the growing volume of user queries imposes substantial deployment costs on LLM-serving providers [13]. For instance, OpenAI reportedly handles up to 12k user queries per second at peak load [49], while its first Azure supercomputer hosted only around 10k GPUs [41], pushing it to rush in more resources to prevent cost spikes [58]. As a result, improving the overall quality of service, particularly under limited token budget constraints, has become a critical priority for LLM-serving systems.

One straightforward approach is to route queries with different features to different LLMs while balancing cost and performance [37, 23, 43]. This idea is based on the observation that different LLMs excel in different domains and have varying cost [43], perfectly aligning with the goal of cost-efficient serving. Existing works typically rely on the model-based predictors [43, 50] or computationally complex calculations (e.g., KNN, similarity-weighted ranking) [43, 23] to predict the optimal LLM to route queries to. While effective in offline scenarios, these methods face significant limitations in practical online routing. They are computationally demanding and introduce additional latency, making them impractical for high-volume, low-latency online environments [43, 23]. They do not easily generalize to dynamic LLM deployments, where any configuration change incurs costly retraining overhead [50, 15, 56]. They are also fundamentally limited in achieving effective online routing under constrained token budgets, as they operate under offline assumptions without accounting for the sequential and uncertain nature of real-world query arrivals [44, 47].

Our work instead proposes the first theoretically grounded online algorithm tailored for high-volume query routing under limited token budget constraints. For each query, we employ Approximate

Nearest Neighbor Search (ANNS) [25, 38] to efficiently estimate its features (performance and cost) for each deployed LLM using a historical dataset. This dataset can be easily collected and maintained by LLM-serving providers from previously served user queries with diverse query types [65], introduces negligible additional deployment overhead, and presents strong adaptivity to different LLM deployments. The routing problem (see Section 1.1 for the formal setup) can be naturally formalized as Mixed-Integer Linear Programming (MILP) that aims to maximize overall performance under token budget constraints. However, solving the global MILP is infeasible in online settings where queries do not arrive simultaneously.

To achieve efficient and near-optimal routing in the realistic sequential setting, we leverage a key observation by looking at the dual problem: At optimality, the dual objective can be fully parameterized by a single dual variable, which directly yields the optimal routing rule. This further motivates us to treat this dual variable as a set of *learnable weights* over LLMs, with the parameterized dual objective serving as the optimization target. Instead of solving the infeasible global dual problem, we adapt it to a partial optimization over a small set of initial observed queries to estimate these weights, which are then applied to guide the routing decisions for subsequent queries. To improve the generalizability of learned weights to future queries, we adopt a random routing strategy for the observed queries, inspired by ideas from PAC learning [39, 11, 10]. In addition, we introduce a control parameter into the MILP objective that does not affect the optimal solution structure but helps to limit performance deviation on future queries.

Regarding efficiency, our algorithm only performs a one-time optimization over a small sample set (i.e., observed queries, typically ≈ 250) and executes practical ANNS per query, making it significantly more computationally efficient than existing methods and well-suited for online routing scenarios. In terms of effectiveness, our theoretical analysis guarantees that the proposed algorithm achieves a competitive ratio of $1 - o(1)$ under mild assumptions. This is further supported by extensive experiments, where our method outperforms all 8 baseline methods across diverse routing settings and three benchmark datasets, achieving an average improvement of $3.55\times$ in performance, $1.85\times$ in cost efficiency, and nearly $4.25\times$ in throughput.

To summarize, our main contributions are:

1. We propose the first **training-free online routing algorithm** tailored for high-volume multi-LLM serving under constrained token budgets.
2. Our algorithm estimates query features via efficient ANNS methods, ensuring **computational scalability**.
3. Unlike prior work, our method operates directly on the auxiliary dataset without any model training, introducing negligible deployment overhead and enabling **deployment scalability** across dynamic LLM deployment configurations.
4. Our algorithm performs a one-time optimization over a small sample set without requiring intensive computational resources, demonstrating **high efficiency**.
5. We provide formal theoretical guarantees showing that our algorithm achieves a competitive ratio of $1 - o(1)$ relative to the offline optimum (which is unknown) under mild assumptions.
6. We conduct extensive experiments on 3 benchmarks against 8 baseline methods, demonstrating that our theoretical guarantees hold in practice and that our algorithm exhibits strong **robustness** and **adaptability** across diverse routing environments.

1.1 Background and Formal Problem Setup

LLM Routing. Existing LLM routing research primarily falls into two paradigms. The first improves response quality while controlling cost, often through ensembling outputs from different LLMs [26, 55] or cascading strategies that query LLMs sequentially by capability [5, 2, 62, 33]. These methods incur high latency and cost due to multiple model calls per query. The second employs learned model-based predictors to estimate the performance or cost for each query and select the optimal LLM [43, 50, 23, 13, 37, 27, 48, 52, 15, 20, 56], but these approaches introduce nontrivial training overhead. Furthermore, adapting them to varying LLM deployment configurations requires retraining, making them unsuitable for dynamic and resource-constrained online routing. Although recent efforts [47, 42, 44] formulate LLM routing as MILP, they struggle in high-volume online settings, where queries arrive sequentially, rather than simultaneously, making the offline optimal infeasible to compute. We complement these works by introducing the first training-free

and efficient online routing method with provable performance guarantees, tailored for high-volume, budget-constrained, and dynamic LLM-serving. See Appendix E for extended related work.

Problem Definition. Consider an LLM-serving system deployed with M types of LLMs. Each LLM type, indexed by $i \in [M] = \{1, \dots, M\}$, is allocated a running token budget B_i (i.e., representing the limited token budgets available per time unit), with a total budget equaling B . Typically, the split of the budget is predetermined before the operation of the system. During each time unit, the system receives a set of queries from various end users, denoted by Q . The goal is to design an online routing strategy $x(\cdot)$ that assigns incoming queries in Q to the available LLMs in a way that maximizes the overall quality of responses, under the budget constraints.

The offline version of this problem can be naturally formulated as the following MILP (Objective 1), where d_{ij} denotes the *performance score* of the response from LLM i to query j , and g_{ij} represents the *token budget consumption* for LLM i processing query j (representing the cost). Specifically, g_{ij} can be further decomposed as $f_i^I \cdot \text{len}(j) + f_i^O \cdot \text{len}(a_{ij})$, where $f_i^I \in \mathbb{R}^M$ and $f_i^O \in \mathbb{R}^M$ represent fixed costs per token during the prefill and decoding stages for LLM i , respectively.

$$\begin{aligned} \max \quad & \sum_{j \in Q} \sum_{i \in [M]} d_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_j g_{ij} x_{ij} \leq B_i \text{ for all } i, \\ & \sum_i x_{ij} \leq 1 \text{ for all } j, \\ & x_{ij} \in \{0, 1\} \end{aligned} \quad (1)$$

However, solving Objective 1 (or a natural relaxation) directly in practical online settings poses several non-trivial challenges: **(i) Inaccessible ground-truth performance and cost:** For any query j , the true performance score d_{ij} and cost g_{ij} are unavailable without accessing the actual LLMs. **(ii) Sequential query arrival under uncertainty:** In practice, queries arrive sequentially rather than simultaneously and must be routed without knowledge of future queries. **(iii) Computational scalability:** High query volume demands routing decisions with low latency and minimal computational overhead. However, methods that rely on solving large-scale MILP or executing computationally intensive model predictors for each incoming query may violate these constraints. **(iv) Deployment scalability:** LLM deployment configurations, such as M or the underlying LLMs, may vary across different environments. Thus, the algorithm must be adaptive to these variations while minimizing adaptation overhead.

Motivated by these challenges, we study the following online routing algorithmic problem: Given a larger set of queries Q arriving in a random sequential order, and a predefined token budget constraint, can we design an online routing algorithm that still achieves a near-optimal cumulative performance? Formally, we aim to ensure $\frac{C_{alg}}{C_{opt}} \geq 1 - o(1)$, where we define $C_{alg} := \sum_j \sum_i d_{ij} \hat{x}_{ij}$ ¹ denotes the total performance of the algorithm with routing results $\hat{x}_{ij}$, and $C_{opt} := \sum_j \sum_i d_{ij} x_{ij}^*$ denotes the offline MILP optimum with the optimal solution x_{ij}^* .

2 Methodology

To tackle the challenges, our algorithm estimates the performance scores and costs efficiently using a historical dataset (Section 2.1), and learns a routing strategy from a small subset of observed queries (of size $\epsilon|Q|$) to guide future query routing (Section 2.2).

2.1 Efficient Performance and Cost Estimation

Our solution to estimate d and g is to leverage a historical dataset D with a specialized data structure that supports efficient similarity-based retrieval. Specifically, let $D = \{j, a_j, d_j, g_j\}_{j=1}^n$, where $a_j \in \mathbb{R}^M$ represents the response vector generated by the M LLMs for query j , and $d_j, g_j \in \mathbb{R}^M$ denotes the corresponding performance scores and costs.

Algorithm 1 Routing with Learned γ^*

- 1: $P \leftarrow \epsilon Q$ (First ϵ -frac. of queries)
 - 2: **for** $j \in P$ **do**
 - 3: Randomly pick $w_j \in \{0\} \cup [M]$
 - 4: Estimate $\hat{d}_{ij}$ and $\hat{g}_{ij}, \forall i \in [M]$
 - 5: **if** $w_j > 0$ **then**
 - 6: Route j to w_j -th LLM
 - 7: Compute $\gamma^* \leftarrow \arg \min_{\gamma} F(\gamma, P)$
 - 8: $Y \leftarrow Q \setminus P$
 - 9: **for** $j \in Y$ **do**
 - 10: Estimate $\hat{d}_{ij}$ and $\hat{g}_{ij}, \forall i \in [M]$
 - 11: Compute $\alpha \hat{d}_{ij} - \hat{g}_{ij} \gamma_i^*, \forall i \in [M]$
 - 12: Route j to $i = \arg \max_i (\alpha \hat{d}_{ij} - \hat{g}_{ij} \gamma_i^*)$
-

¹For simplicity, we use the notation $\sum_j$ to denote $\sum_{j \in Q}$ and $\sum_i$ to denote $\sum_{i \in [M]}$, unless stated otherwise.

For any incoming query $j \in Q$, we apply a classic ANNS method over D in the embedding space, such as DiskANN [25] and HNSW [38], to select the most similar data points for estimation. This yields a set of approximate nearest neighbors for j , denoted by $R_j \subset D$. The estimated performance score $\hat{d}_{ij}$ and cost $\hat{g}_{ij}$ for LLM i are then computed as the mean over these neighbors: $\hat{d}_{ij} = \frac{1}{|R_j|} \sum_{q \in R_j} d_{iq}$, $\hat{g}_{ij} = \frac{1}{|R_j|} \sum_{q \in R_j} g_{iq}$.

One significant advantage of using ANNS compared with other training-based approaches is the ease of updating the historical dataset D to adopt various deployment configurations without requiring model retraining. This is particularly advantageous for LLM-serving providers, as it incurs almost no additional deployment overhead. Online systems can naturally collect and record diverse user queries, which can be directly used to maintain and expand the dataset D . Although collecting ground-truth performance scores and costs typically requires human or automated evaluation, many providers already integrate such mechanisms (e.g., quality feedback from users) in their systems [7]. Furthermore, widely used ANNS algorithms in practice, such as HNSW², use a graph-based indexing structure, making it substantially more efficient in search complexity (typical $O(\log |D|)$ in practice) compared to traditional instance-based methods like exact K-Nearest Neighbor (KNN, $O(|D|)$) [23], achieving significant speedup in search time.

2.2 Online Routing from Observed Queries

With the approximate features $\hat{d}_{ij}$ and $\hat{g}_{ij}$, we propose an online routing algorithm in Algorithm 1.

Approximate LP with Control Parameter. The first step is to approximate the original MILP using the estimated features $\hat{d}_{ij}$ and $\hat{g}_{ij}$. In addition, we introduce a control parameter $\alpha > 0$ into the objective, leading to the formulation in Equation (2). The inclusion of α does not affect the optimal solution structure – it simply scales the objective value. In fact, α acts as a control parameter to ensure generalizability, which is discussed in Section 3.

$$\begin{aligned}
& \max \sum_{j \in Q} \sum_{i \in [M]} \alpha \hat{d}_{ij} x_{ij} & \max \sum_{j \in Q} \sum_{i \in [M]} \alpha \hat{d}_{ij} x_{ij} & \min \sum_{i \in [M]} \gamma_i B_i + \sum_{j \in Q} \beta_j \\
& \text{s.t. } \sum_j \hat{g}_{ij} x_{ij} \leq B_i, \forall i, & \text{s.t. } \sum_j \hat{g}_{ij} x_{ij} \leq B_i, \forall i, & \text{s.t. } \beta_j \geq \alpha \hat{d}_{ij} - \hat{g}_{ij} \gamma_i, \forall i, j, \\
& \sum_i x_{ij} \leq 1, \forall j, & \sum_i x_{ij} \leq 1, \forall j, & \gamma_i \geq 0, \beta_j \geq 0, \forall i, j \\
& x_{ij} \in \{0, 1\}, \forall i, j & x_{ij} \in [0, 1], \forall i, j &
\end{aligned} \tag{2} \tag{3} \tag{4}$$

Dual LP under Relaxation. Let s_{max} denote the max performance score across all queries, and let $\hat{C}_{opt}$ be the *offline approximate optimum* obtained by solving Equation (2) with α removed. When the ratio $\hat{C}_{opt}/s_{max}$ (or C_{opt}/s_{max}) is sufficiently large as $|Q|$ grows, which commonly holds in practice, particularly in high-volume settings, the optimal value to the relaxed LP closely approximates that of the original MILP.³ This is formalized in Lemma 2 and 3 with further discussion in B.1. Based on this observation, we apply LP relaxation to Equation (2), allowing x_{ij} to take fractional values in $[0, 1]$. This yields the relaxed approximate LP in Equation (3) with its dual given in Equation (4).

Routing via Learned γ^* . By complementary slackness, if x is the optimal solution to Equation (3) and (γ, β) is optimal solution to its dual, then $x_{ij} > 0 \Leftrightarrow \beta_j = \max_i (\alpha \hat{d}_{ij} - \hat{g}_{ij} \gamma_i)$. This implies that, at optimality, the dual objective can be expressed as a function parameterized by γ : $F(\gamma) = \sum_i \gamma_i B_i + \sum_j \max_i (\alpha \hat{d}_{ij} - \hat{g}_{ij} \gamma_i)$ (5).

Therefore, ideally, given the offline optimal solution γ , each query j should be routed to the LLM i that maximizes $(\alpha \hat{d}_{ij} - \hat{g}_{ij} \gamma_i)$. While it is infeasible to compute such a global offline solution in an online setting, this formalization motivates us to treat γ as a set of routing weights applied across all LLMs to assist the routing process. Building on this insight, we adapt this idea to the online setting: we learn an estimated set of weights γ^* from the first $P = \epsilon|Q|$ queries. The procedure in Algorithm 1

²We note that many other choices are interchangeable here, e.g. see <https://ann-benchmarks.com>.

³In our experiments, the optimality gap between the relaxed LP and the MILP is only 0.016% on SPROUT.

reflects this idea exactly where its first stage is to learn an optimal γ^* that minimizes dual objective $F(\gamma, P) = \epsilon \sum_i \gamma_i B_i + \sum_{j \in P} \max_i (\alpha \hat{d}_{ij} - \gamma_i \hat{g}_{ij})$ (6).

For each query $j \in P$, the estimated features $\hat{d}_{ij}$ and $\hat{g}_{ij}$ for all models will be calculated and recorded to solve for γ^* . Since this stage requires only a small fraction of the total queries ($\epsilon \ll 1$), random routing is adopted, which may leave some queries in the waiting queue (lines 3, 5-6 in Algorithm 1) to improve the generalizability without degrading overall performance. This approach aligns precisely with the core idea of PAC learning [10, 11], where uniform and unbiased routing ensures the learned weights γ^* generalize to the subsequent routing stage. We formalize this in Lemma 3 and 5. Let the remaining queries be $Y = Q \setminus P$. In the second stage, the learned weights γ^* are used to route each incoming query $j \in Y$. Specifically, each query is directly assigned to the LLM that maximizes the score $(\alpha \hat{d}_{ij} - \gamma_i^* \hat{g}_{ij})$ (line 12 in Algorithm 1). If the selected LLM has exhausted its budget, the query is placed in a queue to await execution.

Compared to existing methods [43, 50, 23], which often involve repeated heavy computations, our algorithm is significantly more efficient. It performs optimization only once over a small subset of queries, making it scalable for deployment in high-volume settings.

3 Theoretical Guarantees

In the following main theorem, we show that our algorithm achieves a competitive ratio close to 1 compared to the offline opt C_{opt} , under natural assumptions (see Section B.1 for discussion).

Theorem 1. *For any given query set Q with random arrival order, Algorithm 1 satisfies $\frac{C_{alg}}{C_{opt}} \geq 1 - O(\epsilon + \delta)$ assuming $\frac{C_{opt}}{s_{max}} \geq \Omega(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)})$, where s_{max} is the maximum performance score obtained for any query, and Φ is an ϵ -net defined over all possible routing strategies $x(\gamma)$.*

To establish this theorem, we introduce the following necessary but mild assumption (see Section B.1 for a detailed discussion) and auxiliary lemmas. The key intuition behind approximating features of one query j using a different query j' is that the error between their feature values remains bounded, as long as their embedding representations are close. We formalize this as follows:

Assumption 1. *For any query j, j' , there exists a value $\eta > 0$ such that if $\|\text{EMB}(j) - \text{EMB}(j')\|_2 \leq \eta$, then $\forall i \in [M]$, it holds that $(1 - O(\delta))d_{ij'} \leq d_{ij} \leq (1 + O(\delta))d_{ij'}$ and $(1 - O(\delta))g_{ij'} \leq g_{ij} \leq (1 + O(\delta))g_{ij'}$, where $\text{EMB}(\cdot)$ represents the embedding function of an embedding model.*

This approximation strategy naturally introduces estimation errors. Thus, it is necessary to quantify the discrepancy between the offline optimum C_{opt} and the offline approximate optimum $\hat{C}_{opt}$. We assume that for any query $j \in Q$, there exists a corresponding feasible set R_j for estimation. This yields the following Lemma 2 (proof in Section B.2).

Lemma 2. *Suppose that $\forall j \in Q$, and $\forall j' \in R_j$, we have $\|\text{EMB}(j) - \text{EMB}(j')\|_2 \leq \eta$, and that Assumption 1 holds. Then, the offline approximate optimum $\hat{C}_{opt}$ satisfies $\left| \frac{\hat{C}_{opt} - C_{opt}}{C_{opt}} \right| \leq O(\delta)$.*

Algorithm 1 yields a set of routing results $\hat{x}$, from which we define the *estimated cumulative performance score* as $C_{est} := \sum_j \sum_i \alpha \hat{d}_{ij} \hat{x}_{ij}$. The results $\hat{x}$ are determined by the learned weights γ^* and the budget limitation, and can be represented by $x(\gamma^*)$. Accordingly, C_{est} can be expressed as $C_{est} = \sum_i \min\{E_i, \sum_j \alpha \hat{d}_{ij} x_{ij}(\gamma^*)\}$, where E_i represents the maximum feasible contribution under the budget B_i , defined as: $E_i := \sum_j^k \alpha \hat{d}_{ij} x_{ij}(\gamma^*)$ with $k = \arg\max_k \sum_j^k \hat{g}_{ij} x_{ij}(\gamma^*)$, s.t. $\sum_j^k \hat{g}_{ij} x_{ij}(\gamma^*) \leq B_i$. We further define the per-LLM estimated performance score $C_{est,i} := \min\{E_i, \sum_j \alpha \hat{d}_{ij} x_{ij}(\gamma^*)\}$. Analyzing C_{est} thus provides a way to evaluate the routing decisions $x(\gamma^*)$. Unfortunately, directly assessing C_{est} is challenging as it has a complex step function with discontinuities. To facilitate analysis, we introduce a relaxed version $\hat{C} := \sum_i \sum_j \alpha \hat{d}_{ij} x_{ij}(\gamma^*)$, which removes the step function present in C_{est} . We further define per-model relaxed estimated performance score $\hat{C}_i := \sum_j \alpha \hat{d}_{ij} x_{ij}(\gamma^*)$, and its counterpart over the observed query set P as $\hat{C}_i(P) := \sum_{j \in P} \alpha \hat{d}_{ij} x_{ij}(\gamma^*)$. Note that for any $j \in P$, we define $x_{ij}(\gamma^*) = 1$ if the index w_j randomly selected in the first stage of the algorithm equals i ; otherwise, $x_{ij}(\gamma^*) = 0$.

By removing the discontinuous step function, the analysis becomes tractable. A key requirement for ensuring the performance of the algorithm is the generalizability of estimated routing weights γ^* on the remaining queries Y . To show this, we first prove that the performance disparity between $\hat{C}(P)$ and its expected value $\epsilon\hat{C}$, i.e., $|\hat{C}(P) - \epsilon\hat{C}|$, is bounded. Since directly bounding this gap over all possible values of γ^* is infeasible due to the infinite parameter space, we use an ϵ -net [60, 11] Φ (formally defined in Definition 1) to reduce to a discrete covering set of routing rules, over which we apply a union bound. We then extend the analysis to arbitrary γ^* values by rounding to Φ . The cumulative error over P is bounded in Lemma 3 (proof in Section B.3):

Lemma 3. *Let Φ be an ϵ -net and assume that $\frac{C_{opt}}{s_{max}} \geq \Omega(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)})$. If $\forall i, |\hat{C}_i(P) - \epsilon\hat{C}_i| \leq z_i$, then $\sum_i z_i \leq O(\epsilon^2 \sqrt{(1+\delta)C_{opt}\hat{C}/\alpha})$.*

The following Lemma demonstrates the generalizability of γ^* on remaining queries Y (proof in Section B.4). This Lemma highlights the role of the control parameter α in our algorithm: it serves as a bridge to connect the cost and performance.

Lemma 4. *If $\forall i$, it holds that $|\sum_{j \in P} \hat{g}_{ij}x_{ij}(\gamma^*) - \epsilon \sum_j \hat{g}_{ij}x_{ij}(\gamma^*)| \leq O(z_i)$, then there exists a control parameter $\alpha > 0$, such that $\forall i$, (1) $|\hat{C}_i(P) - \epsilon\hat{C}_i| \leq z_i$, and (2) $|\hat{C}_i(P) - \epsilon E_i| \leq O(z_i)$.*

This connection further translates the cost constraint (B) to a performance guarantee, which establishes the generalizability of γ^* to the remaining queries Y , as formalized in Lemma 5.

Lemma 5. *Let Φ be an ϵ -net. If $\frac{C_{opt}}{s_{max}} \geq \Omega(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)})$, then $C_{est}(Y) \geq (1 - O(\epsilon))C_{est}$.*

Combining the results above, we can prove our main Theorem 1 (proof in Section B.6).

4 Evaluation

Benchmarks. We use 3 different benchmarks in our experiments: RouterBench [23], SPROUT [50], and Open LLM Leaderboard v2 [16]. RouterBench contains 11 different LLMs, and we randomly sample 10000 queries as the test query dataset and use the remaining data as the historical dataset. Open LLM Leaderboard v2 contains 18 different LLMs, where we similarly sample 10000 queries for the test dataset and use the rest as the historical dataset. SPROUT contains 13 LLMs, and we use the training set as the historical dataset, while the validation and test sets are combined to form the test queries. In the main setting, queries are embedded using bge-base-en-v1.5 [59]. To ensure diversity, we also evaluate with SFR-Embedding-2_R [40] and gte-Qwen2-1.5B-instruct [35].

Baselines. We compare 8 different routing algorithms, classified into two categories: *model-based methods* and *training-free methods*. For model-based methods, following [50], we train two separate Roberta-based models [36] to predict generation performance score and cost, yielding: (i) **Roberta-perf-routing**, routes each query to the model with the highest predicted performance; (ii) **Roberta-cost-routing**, routes each query to the model with the greatest available budget. Training-free methods include: (iv) **Random routing**; (v) **Greedy-perf-routing**, uses ANNS to select the model with the highest predicted performance; (vi) **Greedy-cost-routing**, uses ANNS to select the model with the most available budget; (vii) **KNN-perf-routing** [23], uses KNN to select the model with the highest performance; (viii) **KNN-cost-routing** [23], uses KNN to select the model with the most available budget; (ix) **BatchSplit routing**, groups queries into small batches and solves the LP per batch to determine routing. We adopt HNSW [38] as the main ANNS algorithm and set the number of candidate neighbors ($|R_j|$) to 5 for both ANNS and KNN. For BatchSplit, we use a mini-batch size of 256 to balance LP computation cost with the low-latency requirements of online routing.

Metrics. Three key metrics are used: (1) *Performance*, the total performance score achieved across all test queries; (2) *Performance per Cost*, the ratio of total performance to cost, reflecting cost efficiency; (3) *Throughput*, the total number of queries processed, measuring the processing capacity.

Budget. We aim to improve online routing under limited budgets in high-volume settings. To simulate this, we set the total budget to the minimum cost required for a single model to process all test queries, and vary it by a factor from 0.25 to 2 to evaluate robustness. We consider several strategies for splitting the total budget across models. The main setting uses a cost-efficiency-based split, where the total budget is allocated to each model proportionally to its cost efficiency on the historical dataset D .

Table 1: The main results on RouterBench, SPROUT, and Open LLM Leaderboard v2 are under a total budget equal to the cost of the cheapest model, split across models based on cost efficiency. Here, Perf represents the *Performance*, PPC represents *Performance per Cost*, Tput represents *Throughput*, and RP denotes *Relative Performance* compared with offline approximate optimum ($\hat{C}_{opt}$).

Algorithm	RouterBench					SPROUT					Open LLM Leaderboard v2				
	Perf	Cost	PPC	Tput	RP	Perf	Cost	PPC	Tput	RP	Perf	Cost	PPC	Tput	RP
Random	1384.25	0.427	3243.25	3276	43.10%	2827.6	0.72	3927.29	4742	47.61%	953.0	0.741	1284.37	2877	49.89%
Greedy-Perf	1012.1	0.27	3742.379	1687	31.52%	764.9	0.406	1881.742	1083	12.88%	553.0	0.499	1107.91	1189	28.95%
Greedy-Cost	1626.25	0.46	3534.46	4061	50.64%	3934.7	0.849	4630.41	6789	66.25%	1051.0	0.766	1371.30	3164	55.02%
KNN-Perf	1005.1	0.27	3720.58	1677	31.3%	769.6	0.407	1888.46	1084	12.96%	556.0	0.498	1114.29	1194	29.11%
KNN-Cost	1592.05	0.46	3454.04	4027	49.58%	3905.1	0.85	4593.37	6709	65.75%	991.0	0.766	1293.07	3172	51.88%
BatchSplit	1838.05	0.458	4005.93	3903	57.24%	3975.5	0.83	4784.49	6221	66.94%	1059.0	0.76	1392.07	3099	55.44%
Roberta-Perf	154.5	0.077	2019.00	190	4.81%	458.9	0.283	1621.64	536	7.73%	153.0	0.207	738.21	283	8.01%
Roberta-Cost	481.4	0.129	3738.88	1292	14.99%	3996.2	0.848	4709.22	6765	67.29%	1044.0	0.766	1362.53	3173	54.66%
Ours	2718.6	0.447	6075.58	5195	84.66%	4513.05	0.815	5536.74	7475	75.99%	1465.0	0.711	2060.3	3692	76.7%
<i>Offline Oracle (Algorithm Upper Bounds Reference)</i>															
Approx Optimum($\hat{C}_{opt}$)	3211.35	0.46	6975.16	6225	100%	5938.99	0.85	6986.45	8781	100%	1910.0	0.765	2493.66	4319	100%
Optimum (C_{opt})	6376.9	0.46	13865.62	6436	198.57%	11934.4	0.848	14060.34	12336	200.94%	4688.0	0.763	6143.64	4688	245.44%

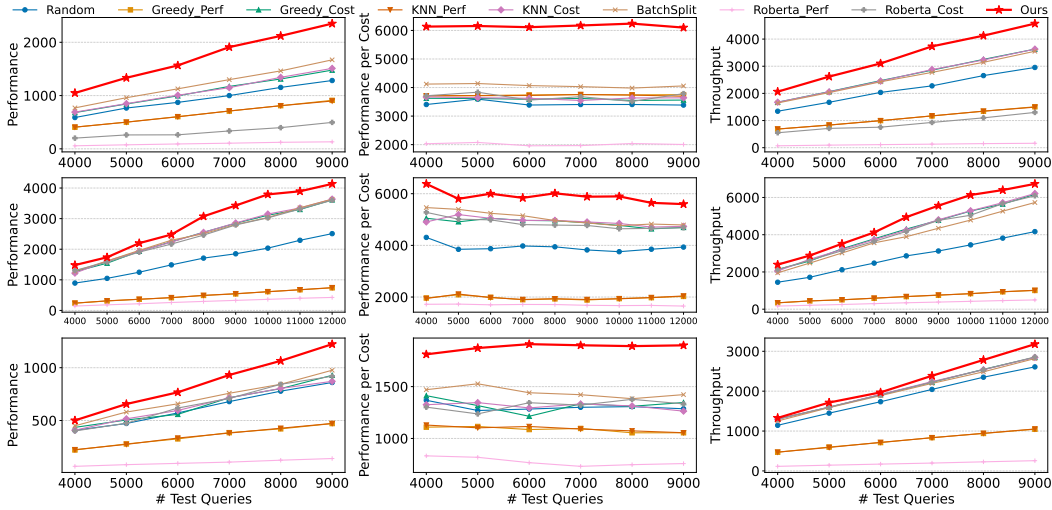


Figure 1: Results with test query volume varying from 4000 to 9000 (12000). Rows correspond to different datasets: RouterBench (top), SPROUT (middle), and Open LLM Leaderboard v2 (bottom).

To assess robustness, we also consider uniform, random, extreme, cost-based, and performance-based splits. For the random split, budgets are randomly assigned and averaged over 100 runs. In the extreme split, 80% of the budget is allocated to the h least cost-efficient models ($h = 1$ to 5), and the remaining 20% to the others. For additional experimental setup details, see Section A.

Main Results. Table 1 presents the main results under the 3 benchmarks, using $\alpha = 0.0001$ and $\epsilon = 0.025$ for our algorithm, with the maximum available test queries, and historical data in the evaluation. Our algorithm consistently outperforms all 8 baselines in performance, cost efficiency, and throughput. On average, it exceeds all baselines by $3.55\times$ in performance, $1.85\times$ in cost efficiency, and nearly $4.25\times$ in throughput. Even against the strongest baseline, BatchSplit, our algorithm still achieves with **33%** higher performance, **38%** better cost efficiency, and **24%** higher throughput across all benchmarks. We further compare our method to the offline approximate oracle ($\hat{C}_{opt}$), and find that it achieves **75.99%** to **84.66%** of the approximate oracle’s performance. These results align closely with our theoretical guarantees, further validating the effectiveness of our algorithm.

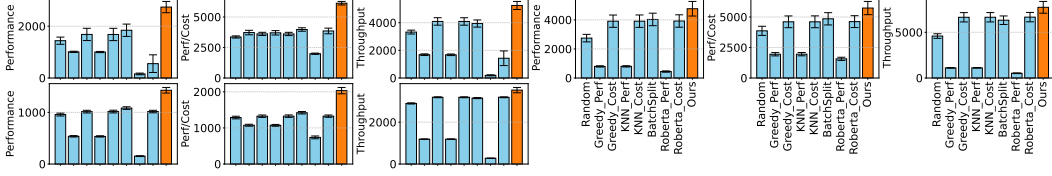


Figure 2: Results under 100 random query orders. (Left to right): the first three subfigures show results on RouterBench, the next three on SPROUT, and the last three on Open LLM Leaderboard v2.

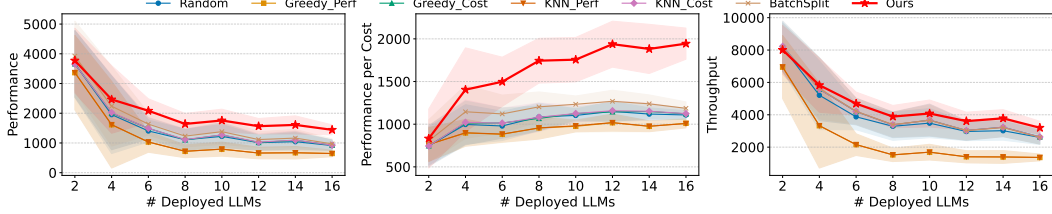


Figure 3: Results on Open LLM Leaderboard v2 when varying LLM deployment configurations.

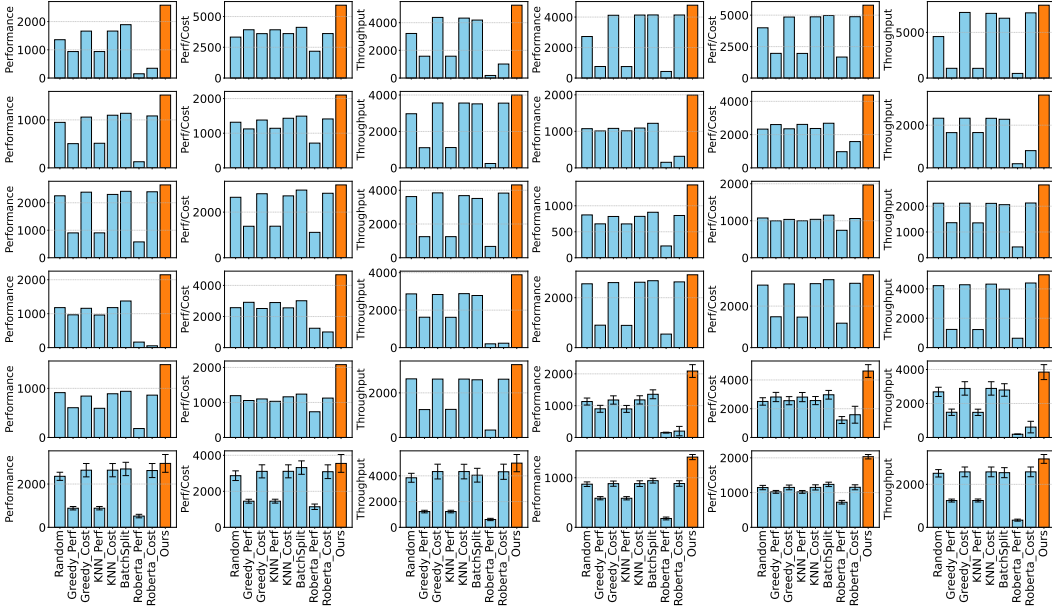


Figure 4: Results under different budget splitting strategies. Each group of 9 subfigures corresponds to one strategy: (a) cost-based split (subfigures 1–9), (b) performance-based split (10–18), (c) uniform split (19–27), and (d) random split (28–36). Within each group, the first three subfigures correspond to RouterBench, the next three to SPROUT, and the last three to Open LLM Leaderboard v2.

4.1 Robustness

Query Volume. We vary the number of test queries from 4000 to 12000, serving as different traffic volumes. Figure 1 clearly shows that our method consistently outperforms all baselines across all benchmarks and metrics. It scales gracefully, maintaining the highest performance, cost efficiency, and throughput as the query volume increases from low to extremely high loads. Notably, across all three benchmarks, the performance gap between our method and the strongest baseline (BatchSplit) widens with increasing load, reaching about 50% higher performance on RouterBench at maximum volume. These results highlight the strong robustness of our approach under varying query volumes.

Query Arrival Order. We evaluate the robustness of our algorithm under varying query arrival orders. Specifically, we independently shuffle test queries 100 times to simulate realistic, unpredictable online environments. As shown in Figure 2, our method consistently outperforms all baselines across metrics and benchmarks under these random permutations, demonstrating strong robustness. We further

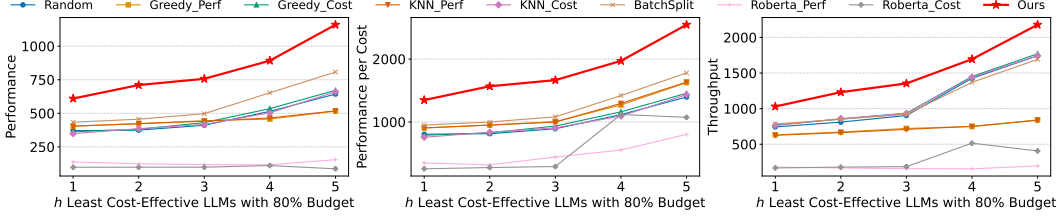


Figure 5: Results on RouterBench under the extreme budget split.

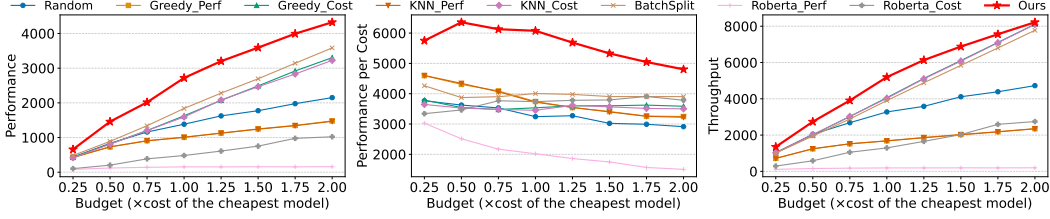


Figure 6: Results on RouterBench with budget from 0.25 to $2\times$ the cost of the cheapest model.

evaluate a worst-case adversarial setting, where expensive queries arrive first with results provided in Section C.1, showing that our algorithm still maintains strong performance.

Scalability to LLM Deployments. We assess robustness under varying deployment configurations by randomly selecting 2 to 16 LLMs from the Open LLM Leaderboard v2 and repeating each experiment 10 times for diversity. As shown in Figure 3, our algorithm consistently achieves strong performance across configurations, demonstrating strong robustness and adaptability to diverse LLM deployments. Additional results on RouterBench and SPROUT (see Section C.2) further confirm its scalability.

Budget Split. We extend the default cost-efficiency-based split to five alternative strategies. As shown in Figure 4, our algorithm consistently outperforms all baselines across all benchmarks and metrics under cost-based, performance-based, uniform, and random splits. Even under the extreme split scenario (Figure 5), it still achieves strong results. Notably, when 80% of the budget is allocated to a single model ($h = 1$), our algorithm achieves nearly $2\times$ the performance of the strongest baseline (BatchSplit) on RouterBench. Additional results on SPROUT and Open LLM Leaderboard v2 (see Section C.3) further highlight its robustness across diverse budget allocation schemes.

Total Budget. We vary the total budget B from 0.25 to $2\times$ the cost of the cheapest model. Results shown in Figure 6 demonstrate that our algorithm consistently outperforms all baselines across metrics on RouterBench as the budget increases. Even under extremely limited budgets, our algorithm can still achieve the best performance. Additional results on SPROUT and Open LLM Leaderboard v2 (see Section C.4) further highlight its robustness and adaptability across diverse budget constraints.

Routing Overhead & Quality of Historical Data. We evaluate the routing latency of Algorithm 1 under varying query volumes. As shown in Appendix Table 7, our method incurs negligible decision overhead compared to existing training-free baselines. To assess robustness to data quality, we further consider a noisy setting with label perturbations and an out-of-distribution setting with distribution shift. Results in Appendix Table 8 indicate that our algorithm consistently outperforms all baselines in both settings. See Section C.5 for detailed results.

Historical Data & Search Candidates. Further results in Appendix C.6 show that our algorithm remains robust under varying historical data sizes and different numbers of ANNS/KNN candidates.

4.2 Ablation Studies

(1) Impact of Embedding Models. We evaluate the impact of different embedding models used in ANNS and observe that our algorithm consistently outperforms baselines across all benchmarks (see Appendix Figure 13), demonstrating its robustness to embedding choice. **(2) Impact of α and ϵ .** We evaluate sensitivity to α and ϵ , as shown in Appendix Figure 14. Performance decreases with larger α , with the best result at 0.0001, aligned with Lemma 4. For ϵ , performance increases initially, reaches the peak, and then declines with further increase. See Section C.7 for details.

5 Limitations & Conclusion

Multi-Factor Routing. In more complex LLM-serving scenarios, additional factors such as query traffic may need to be considered. However, the primary goal of this work is to develop a cost-efficient online routing algorithm, a problem that has received growing attention in recent literature [43, 23, 50]. To isolate this core algorithmic problem, we focus solely on two of the most widely studied factors: performance and cost [43, 23, 50], without explicitly modeling other routing considerations. Those routing factors can be broadly categorized into two types. (i) **Query-intrinsic features**, which are independent of time and query order, such as per-model response quality and cost, can be seamlessly incorporated into our algorithm by adding additional constraints to the original MILP formulation. Each added constraint introduces a new linear term with a corresponding dual variable in the dual objective, and these dual variables together serve as learnable routing weights for future routing. (ii) In contrast, **operational-state features**, which depend on dynamic system conditions (e.g., query traffic or system load), fall outside the scope of our current formulation. These factors are more aligned with a related but orthogonal problem: load balancing under traffic constraints. Nonetheless, we believe our algorithm offers a natural foundation for future extensions that incorporate such traffic-aware adaptations. Further discussion is provided in Section D.

Conclusion. This work presents the first efficient, training-free online routing algorithm for high-volume, budget-constrained LLM serving. Our algorithm uses ANNS over historical data to efficiently estimate query features and performs a one-time optimization on a small set of observed queries to learn routing weights that ensure strong performance and generalizability, and efficiency. We theoretically guarantee a competitive ratio of $1 - o(1)$ under mild assumptions, validated by extensive experiments on 3 benchmarks against 8 baselines. Our method consistently outperforms all baselines, underscoring strong effectiveness and robustness in diverse online routing scenarios.

References

- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [4] Niv Buchbinder, Kamal Jain, and Joseph Naor. Online primal-dual algorithms for maximizing ad-auctions revenue. In *European Symposium on Algorithms*, pages 253–264. Springer, 2007.
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2023.
- [8] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

- [9] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>, 9, 2021.

- [12] Steven Diamond and Stephen Boyd. Cvxpy: A python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

- [14] Jon Feldman, Nitish Korula, Vahab Mirrokni, Shanmugavelayutham Muthukrishnan, and Martin Pál. Online ad assignment with free disposal. In *International workshop on internet and network economics*, pages 374–385. Springer, 2009.

- [16] C. Fourrier, N. Habib, A. Lozovskaya, K. Szafer, and T. Wolf. Open llm leaderboard v2. https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard, 2024.

- [18] Robert Friel, Masha Belyi, and Atindriyo Sanyal. Ragbench: Explainable benchmark for retrieval-augmented generation systems. *arXiv preprint arXiv:2407.11005*, 2024.
- [19] Gagan Goel and Aranyak Mehta. Online budgeted matching in random input models with applications to adwords. In *SODA*, volume 8, pages 982–991, 2008.
- [20] Surya Narayanan Hari and Matt Thomson. Tryage: Real-time, intelligent routing of user prompts to large language models. *arXiv preprint arXiv:2308.11601*, 2023.

- [23] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing system. *arXiv preprint arXiv:2403.12031*, 2024.
- [24] Qi Huangfu and JA Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

- [27] Wittawat Jitkrittum, Harikrishna Narasimhan, Ankit Singh Rawat, Jeevesh Juneja, Zifeng Wang, Chen-Yu Lee, Pradeep Shenoy, Rina Panigrahy, Aditya Krishna Menon, and Sanjiv Kumar. Universal model routing for efficient llm inference. *arXiv preprint arXiv:2502.08773*, 2025.
- [28] Bala Kalyanasundaram and Kirk R Pruhs. An optimal deterministic algorithm for online b-matching. *Theoretical Computer Science*, 233(1-2):319–325, 2000.
- [31] Thomas Kesselheim, Klaus Radke, Andreas Tönnis, and Berthold Vöcking. An optimal online algorithm for weighted bipartite matching and extensions to combinatorial auctions. In *European symposium on algorithms*, pages 589–600. Springer, 2013.
- [34] Kay Lehnert. Ai insights into theoretical physics and the swampland program: A journey through the cosmos with chatgpt. *arXiv preprint arXiv:2301.08155*, 2023.
- [35] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023.
- [36] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.
- [38] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.
- [39] Aranyak Mehta, Amin Saberi, Umesh Vazirani, and Vijay Vazirani. Adwords and generalized online matching. *Journal of the ACM (JACM)*, 54(5):22–es, 2007.
- [40] Rui Meng, Ye Liu, Shafiq Rayhan Joty, Caiming Xiong, Yingbo Zhou, and Semih Yavuz. Sfr-embedding-2: Advanced text embedding with multi-stage training, 2024.
- [41] Microsoft News Center. Microsoft unveils a 10 k-gpu azure supercomputer for openai. <https://news.microsoft.com/source/features/ai/openai-azure-supercomputer/>, 2020.

- [42] Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. Routoo: Learning to route to large language models effectively. *arXiv preprint arXiv:2401.13979*, 2024.

- [46] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

- [49] Shubham Singh. Chatgpt statistics (2025): Dau & mau data worldwide. <https://www.demandsage.com/chatgpt-statistics/>, 2025.
- [50] Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, Prattyush Mangal, Mírian Silva, Onkar Bhardwaj, Mikhail Yurochkin, and Subha Maity. Carrot: A cost aware rate optimal router. *arXiv preprint arXiv:2502.03261*, 2025.

- [53] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, 2023.
- [54] Teknium. Openhermes-2.5. <https://huggingface.co/datasets/teknium/OpenHermes-2.5>, 2023.

- [58] Kyle Wiggers. Openai ceo sam altman says the company is 'out of gpus'. <https://techcrunch.com/2025/02/27/openai-ceo-sam-altman-says-the-company-is-out-of-gpus/>, 2025.
- [60] Haike Xu, Piotr Indyk, and Sandeep Silwal. A bi-metric framework for efficient nearest neighbor search. In *The 1st Workshop on Vector Databases*, 2025.
- [64] Qiang Zhang, Keyan Ding, Tianwen Lv, Xinda Wang, Qingyu Yin, Yiwen Zhang, Jing Yu, Yuhao Wang, Xiaotong Li, Zhuoyi Xiang, et al. Scientific large language models: A survey on biological & chemical domains. *ACM Computing Surveys*, 57(6):1–38, 2025.
- [66] Ciyou Zhu, Richard H Byrd, Peihuang Lu, and Jorge Nocedal. Algorithm 778: L-bfgs-b: Fortran subroutines for large-scale bound-constrained optimization. *ACM Transactions on mathematical software (TOMS)*, 23(4):550–560, 1997.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

Answer: [\[Yes\]](#)

Justification: Yes, the method, theoretical analysis, and experiment sections justify all the claims made in the abstract and introduction. Additional clarifications are also provided in the Appendix.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Yes, we outline the limitation of our method in Section 5.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[Yes\]](#)

Justification: Yes, we provide the full set of assumptions, lemmas, and the main theorem in Section 3, along with detailed discussions and complete proofs in Section B.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: Yes, we disclose all information needed to reproduce the main experimental results of the paper in Section 4 and Section A.

Guidelines:

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: Yes, all the models and datasets we work with in the paper are open-source, and we have submitted our code in the Supplementary Material.

Guidelines:

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Yes, all experimental details are clarified in Section 4 and Section A.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: Yes, we report error bars in all experiments involving randomness, such as those with randomized query arrival orders, to capture the variability across runs and evaluate the statistical robustness of the results.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [\[Yes\]](#)

Justification: We provide the computer resources needed for all experiments in Section A.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [\[Yes\]](#)

Justification: Yes, the broader impacts of the work are discussed in Section F.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: Our paper does not pose any such risks.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All the existing models and datasets are appropriately cited.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: We introduce a new algorithm in the paper and provide its implementation along with detailed documentation to support reproducibility.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[NA\]](#)

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[NA\]](#)

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [NA]

Justification: This research does not involve LLMs as any important, original, or non-standard components.

Guidelines:

A More Experiment Details

Table 2: Benchmark details for RouterBench, SPROUT, and Open LLM Leaderboard v2 in our experiments.

(a) RouterBench		(b) SPROUT		(c) Open LLM Leaderboard v2	
Dataset	Size	Dataset	Size	Dataset	Size
MMLU [21]	14042	MATH Lvl 1-5 [22]	9884	MMLU-PRO [57]	12032
Hellaswag [63]	10042	MMLU-PRO [57]	11786	MUSR [51]	756
GSM8K [9]	7450	GPQA [45]	541	MATH Lvl 1-5 [22]	1324
ARC Challenge [8]	1470	MUSR [51]	748	GPQA [45]	1192
Winogrande [46]	1267	RAGBench [18]	1827	BBH [53]	5761
MBPP [3]	427	Openhermes 2.5 [54]	19455	Total	21065
MT-Bench [65]	80	Total	44241	–	–
Chinese	785	–	–	–	–
Consensus Summary	362	–	–	–	–
Bias Detection	285	–	–	–	–
Test Match	3	–	–	–	–
Accounting Audit	30	–	–	–	–
Abstract2title	254	–	–	–	–
Total	36497	–	–	–	–

Benchmarks. We evaluate our method on three benchmarks: RouterBench (zero-shot version) [23], SPROUT [50], and Open LLM Leaderboard v2 [16]. All three benchmarks are constructed from multiple dataset sources. Table 2 summarizes the query types in each benchmark, while Table 3 details the LLMs used and their corresponding token costs.

For RouterBench, we identify 13 different data sources in a total of 36497 samples – 6 more than those reported in the original paper – spanning a diverse range of query domains. This benchmark includes 11 different LLMs. Since it does not provide a predefined train/test split, we randomly sample 10000 queries as the test query set and treat the remaining queries as historical data. Note that we do not report per-token costs for models in RouterBench, as [23] does not provide this information. Instead, the benchmark includes precomputed costs for each query across all 11 LLMs, which we directly use in our experiments.

For SPROUT, it consists of queries from 6 different datasets, covering different query domains, such as math and RAG. It contains a total of 44241 samples across 13 LLMs. We use the provided training set as the historical dataset, and combine the validation and test sets to construct the test query set.

For Open LLM Leaderboard v2, we follow the same setup and data processing as [50], resulting in 21065 samples. The processed benchmark includes 5 different datasets, with each evaluating different aspects of LLM capabilities. This benchmark uses 18 different LLMs, and we randomly sample 10000 queries as the test set and use the remainder as the historical data. Since most evaluations in Open LLM Leaderboard v2 are likelihood-based, the cost is essentially determined by the length of the input. Following [50], we report the cost per input token in Table 3 and compute the total cost of each query based on input token counts.

In the main setting, the queries are embedded using `bge-base-en-v1.5` [59]. For the diversity consideration, we additionally evaluate with two different embedding models: `SFR-Embedding-2_R` [40] and `gte-Qwen2-1.5B-instruct` [35].

Baselines. We compare 8 different routing algorithms, categorized into two categories: *model-based methods* and *training-free methods*.

For Model-based methods, we follow the training approach in [50] and train two separate Roberta-based models [36] to predict the performance and cost of each query. The performance prediction model is classification-based, aiming to select the optimal LLM with the highest expected perfor-

Table 3: Model lists and token costs (\$/1M tokens) for RouterBench(left), SPROUT (middle), and Open LLM Leaderboard v2 (right).

(a) RouterBench		(b) SPROUT		(c) Open LLM Leaderboard v2	
Models		Models	Input Cost Output Cost	Models	Cost
WizardLM-13B-V1.2		claude-3-5-sonnet-v1	3.00 15.00	Mixtral-8x7B-DPO	0.6
claude-instant-v1		titan-text-premier-v1	0.50 1.50	Yi-34B-Chat	0.8
claude-v1		openai-gpt-4o	2.50 10.00	QwQ-32B-Preview	1.2
claude-v2		openai-gpt-4o-mini	0.15 0.60	Qwen2-72B-Instruct	0.9
gpt-3.5-turbo-1106		granite-3-2b-instruct	0.10 0.10	Qwen2.5-7B-Instruct	0.3
gpt-4-1106-preview		granite-3-8b-instruct	0.20 0.20	Qwen2.5-72B-Instruct	1.2
code-llama-instruct-34b		llama-3-1-70b-instruct	0.90 0.90	WizardLM-2-8x22B	1.2
llama-2-70b-chat		llama-3-1-8b-instruct	0.20 0.20	deepseek-llm-67b-chat	0.9
mistral-7b-chat		llama-3-2-1b-instruct	0.06 0.06	gemma-2-27b-it	0.8
mixtral-8x7b-chat		llama-3-2-3b-instruct	0.06 0.06	gemma-2-9b-it	0.3
Yi-34B-Chat		llama-3-3-70b-instruct	0.90 0.90	gemma-2b-it	0.1
–		mixtral-8x7b-instruct	0.60 0.60	Llama-2-13b	0.3
–		llama-3-405b-instruct	3.50 3.50	Meta-Llama-3.1-70B	0.9
–		–	– –	Mistral-7B-Instruct-v0.1	0.2
–		–	– –	Mistral-7B-Instruct-v0.2	0.2
–		–	– –	Mistral-7B-Instruct-v0.3	0.2
–		–	– –	Mixtral-8x7B-Instruct-v0.1	0.6
–		–	– –	Llama-3.1-Nemotron-70B	0.9

mance, while the cost prediction model is regression-based and estimates the expected cost of an incoming query. This results in 2 different baselines:

- **Roberta-perf-routing**, routes each query to the model with the highest predicted performance;
- **Roberta-cost-routing**, routes each query to the model with the greatest available budget.

Training-free methods include 6 baselines:

- **Random routing**, randomly selects a model for each query;
- **Greedy-perf-routing**, uses ANNS to select the model with the highest predicted performance;
- **Greedy-cost-routing**, uses ANNS to select the model with the most available budget;
- **KNN-perf-routing** [23], uses KNN to select the model with the highest performance;
- **KNN-cost-routing** [23], uses KNN to select the model with the most available budget;
- **BatchSplit routing**, groups incoming queries into small batches and solves the linear programming (LP) per batch to determine routing.

Note that in the online setting, the true remaining budget of each model is not directly observable during the routing process, as it depends on the actual cost incurred after the query response is generated. Therefore, all cost-based methods and BatchSplit rely on predicted costs, which are obtained from predictive models, ANNS, or KNN, to estimate the remaining available budget and make routing decisions.

We adopt HNSW [38] as the main ANNS algorithm and set the number of candidate neighbors ($|R_j|$) to 5 for both ANNS and KNN. To assess the robustness of our algorithm under varying candidate set sizes, we further vary this number to 3, 7, and 10. However, many alternative methods, such as DiskANN [25], are interchangeable here, as listed in <https://ann-benchmarks.com>. For BatchSplit, we use a mini-batch size of 256 to balance LP computation cost with the low-latency requirements of the practical online routing.

Metrics. In our experiments, we consider three key evaluation metrics: (1) *Performance*, the overall performance score achieved by processing all test queries under the given budget constraints; (2) *Performance per Cost*, the ratio of total performance to the corresponding cost, reflecting overall

Table 4: Average cost and performance of models in RouterBench on historical data.

Model	Cost	Perf	Perf/Cost	(Perf/Cost) ^{0.5}
WizardLM-13B-V1.2	7.27e−05	0.432	5944	77.10
claude-instant-v1	2.32e−04	0.598	2581	50.80
claude-v1	2.14e−03	0.631	295	17.18
claude-v2	2.41e−03	0.636	264	16.24
gpt-3.5-turbo-1106	2.42e−04	0.617	2546	50.45
gpt-4-1106-preview	3.28e−03	0.781	238	15.43
code-llama-instruct-34b	1.71e−04	0.203	1182	34.38
llama-2-70b-chat	2.02e−04	0.328	1627	40.34
mistral-7b-chat	4.56e−05	0.308	6768	82.27
mixtral-8x7b-chat	1.34e−04	0.550	4098	64.01
Yi-34B-Chat	1.85e−04	0.648	3503	59.19

Table 5: Average cost and performance of models in SPROUT on historical data.

Model	Cost	Perf	Perf/Cost	(Perf/Cost) ^{0.5}
claude-3-5-sonnet-v1	7.65e−03	0.827	108	10.39
titan-text-premier-v1	5.64e−04	0.579	1027	32.04
openai-gpt-4o	4.92e−03	0.846	172	13.11
openai-gpt-4o-mini	3.40e−04	0.808	2378	48.76
granite-3-2b-instruct	8.54e−05	0.553	6473	80.46
granite-3-8b-instruct	1.50e−04	0.659	4403	66.36
llama-3-1-70b-instruct	7.17e−04	0.810	1130	33.62
llama-3-1-8b-instruct	2.43e−04	0.690	2838	53.27
llama-3-2-1b-instruct	6.67e−05	0.460	6904	83.09
llama-3-2-3b-instruct	6.47e−05	0.629	9722	98.60
llama-3-3-70b-instruct	5.52e−04	0.804	1457	38.17
llama-3-405b-instruct	2.01e−03	0.776	385	19.63
mixtral-8x7b-instruct	3.74e−04	0.616	1648	40.59

cost efficiency; (3) *Throughput*, the total number of queries successfully processed, indicating the processing capacity. We evaluate *Throughput* as a key metric because our goal is to design an effective online routing algorithm for high-volume settings with limited token budgets. In these scenarios, the available budget may not suffice to serve all incoming queries within a given time unit. Therefore, *Throughput* reflects how efficiently an algorithm utilizes the available budget to maximize the number of queries successfully served during that current time unit.

Budget. Our algorithm aims to improve routing performance under limited budgets in high-volume settings. To simulate this high-volume, budget-constrained setting, we define the total budget based on the minimal cost required for a single model to process all test queries in the benchmark. In the main setting, the budget is set to this minimal value. To evaluate the robustness of our algorithms, we scale the budget by a factor ranging from 0.25 to 2.

We consider multiple strategies for splitting the total budget across models. In the main setting, we adopt a cost-efficiency-based split strategy. As shown in Table 4, 5, and 6, we report the average performance score, cost, and cost efficiency (Perf/Cost) of each model on the historical data across 3 benchmarks. We observe a substantial disparity in cost efficiency across models. For instance, for SPROUT, the most efficient model achieves a cost efficiency of 9722, while the least efficient only reaches 108, nearly a 100× difference. Directly allocating the budget based on cost efficiency is thus highly imbalanced, as it can result in nearly all resources being allocated to a single model.

Table 6: Average cost and performance of models in Open LLM Leaderboard v2 on historical data.

Model	Cost	Perf	Perf/Cost	(Perf/Cost) ^{0.5}
Yi-34B-Chat	6.57e−04	0.428	652	25.53
Mixtral-8x7B-DPO	4.78e−04	0.401	839	28.97
QwQ-32B-Preview	8.90e−04	0.552	621	24.91
Qwen2-72B-Instruct	6.67e−04	0.562	842	29.02
Qwen2.5-72B-Instruct	8.90e−04	0.561	630	25.10
Qwen2.5-7B-Instruct	2.22e−04	0.420	1887	43.44
WizardLM-2-8x22B	9.85e−04	0.491	499	22.34
deepseek-llm-67b-chat	7.05e−04	0.413	586	24.21
gemma-2-27b-it	6.13e−04	0.462	753	27.43
gemma-2-9b-it	2.30e−04	0.419	1826	42.72
gemma-2b-it	7.66e−05	0.191	2489	49.89
Llama-2-13b	2.47e−04	0.227	919	30.31
Meta-Llama-3.1-70B	6.44e−04	0.548	852	29.18
Mistral-7B-Instruct-v0.1	1.43e−04	0.258	1806	42.50
Mistral-7B-Instruct-v0.2	1.64e−04	0.311	1894	43.52
Mistral-7B-Instruct-v0.3	1.64e−04	0.336	2044	45.21
Mixtral-8x7B-Instruct-v0.1	4.92e−04	0.379	770	27.74
nvidia/Llama-3.1-Nemotron-70B	7.39e−04	0.506	686	26.19

Therefore, we adopt a smoothed version that allocates the budget proportionally to the square root of each model’s cost efficiency on the historical data, i.e., according to $(\frac{\text{PERF}}{\text{COST}})^{0.5}$.

In our robustness evaluations, we explore several alternative budget splitting strategies, including uniform split, random split, extreme split, cost-based split, and performance-based split. In the cost-based split, the budget is allocated inversely proportional to the average cost. However, due to the extreme cost imbalance across models (as also observed in the cost-efficiency-based split), we also adopt a smoothed variant that splits the budget proportional to $(\frac{1}{\text{COST}})^{0.5}$. In the performance-based split, the budget is directly assigned in proportion to the average performance scores of the models. In the random split, the total budget is randomly allocated across LLMs, and the experiment is repeated 100 times to account for variability. In the extreme split, 80% of the budget is assigned to the h least cost-efficient models, and the remaining 20% is uniformly distributed among the others, where h ranges from 1 to 5.

Optimization Implementation. We implement all optimization codes using the CVXPY [12] package. For the one-time optimization step in our algorithm, we use the L-BFGS-B solver [66]. For BatchSplit, which involves solving a linear program for each batch, we adopt the HIGHS solver [24].

Devices. All experiments are conducted on a machine equipped with 16 CPUs and 32GB of memory. For training the Roberta models used in the model-based baselines, we use two NVIDIA H200 GPUs.

B Theoretical Proofs

Definition 1 (ϵ -Net [11]). Given a parameter $\epsilon > 0$ and routing rule $x(\gamma)$, a set $\Phi \subseteq [0, 1]^M$ is an ϵ -net if, for any $\gamma \in [0, 1]^M$, there exists $\gamma' \in \Phi$ such that $\forall i, j, |x_{ij}(\gamma) - x_{ij}(\gamma')| \leq \epsilon$.

B.1 Discussion on Our Theoretical Assumptions

We introduce several mild assumptions in our theoretical analysis and provide justifications for each below.

Random arrival order. We assume that the queries arrive in a random order, i.e., they can be picked adversarially, but their order is randomly permuted. Note that this is a weaker requirement

than assuming the queries are sampled i.i.d. from an unknown distribution. This is because if the queries are sampled i.i.d., then they also satisfy the random order requirement, and thus our assumption is weaker. Furthermore, the random order model is also a popular assumption in many other online algorithms [39, 11], since it typically allows one to go beyond worst-case hardness and obtain meaningful theoretical guarantees. In practice, we perform robustness studies by changing the query order and observe that our algorithm still outperforms baselines; see Section 4.1.

“ C_{opt} is enough large” is practical. In Theorem 1, we assume that $\frac{C_{opt}}{s_{max}} \geq \Omega(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)})$, which is naturally aligned with high-volume practical settings. This is because C_{opt} depends on the total number of queries $|Q|$, whereas our lower bound assumption scales as a function of M , the number of LLMs. Typically, M can be thought of as a constant while $|Q|$, the total number of queries, should grow over time. Indeed, in all of our benchmarks, we observed that for *every query* j , the optimal solution x_{ij}^* (that is, the performance score achieved by the optimal solution on query j) satisfies $\sum_{i \in [M]} d_{ij} x_{ij}^* \geq 0.462$ consistently on Open LLM Leaderboard v2, 0.639 on RouterBench, and 0.887 on SPROUT, meaning that in practice, C_{opt} scales as $\Omega(Q)$ (linear in Q). On the other hand, for constant ϵ and α (e.g. we use $\alpha = 0.0001$, $\epsilon = 0.025$ in our main setting), our required lower bound is a polynomial of M (more specifically, it is close to a quadratic since Φ may depend exponentially in M but it is inside a logarithm). However, we think of M as a constant. For example, $M \leq 18$ in all of our experiments. Even if M is large, it is reasonable to assume that it is orders of magnitude smaller than $|Q|$ (in our experiments, Q is on the order of thousands to tens of thousands, and in real-world systems, it can be much larger).

Our assumption is further corroborated by the relaxation used in Section 2.1, where if $\frac{C_{opt}}{s_{max}}$ is sufficiently large, the original MILP can be closely approximated by its fractional LP relaxation with only a negligible optimality gap. We validate this in our main experiments over three benchmarks with the observed optimality gaps being: (i) 0.016% on SPROUT, (ii) 0.086% on RouterBench, and (iii) 0.3% on Open LLM Leaderboard v2.

Lastly, we remark that similar reasonable lower-bound assumptions have been made in prior work on the online matching problem [19, 11, 39], where it is commonly assumed that bids for the items are small compared to the total budget.

“Estimating query features via similarity” is natural. The core intuition behind most previous literature on predictive LLM routing [23, 43, 56, 50, 52], which trains a model-based predictor on auxiliary datasets to estimate performance and cost of future queries, aligns with Assumption 1. Specifically, these approaches assume that similar queries share similar routing-relevant features. If no such relationship exists, then historical data would be uninformative, and it would be impossible to leverage historical data for effective future routing. In this case, no meaningful inference can be made without directly querying the LLMs, and routing decisions degenerate to random selection.

One may argue that useful routing information can still be obtained by querying only a subset of LLMs. This idea is echoed in works that adopt cascading strategies, where LLMs are queried sequentially based on their observed capabilities on historical data [2, 5]. However, these approaches also fundamentally rely on a related assumption: LLMs that perform well on historical data or benchmarks are likely to perform well on future queries. Furthermore, these methods introduce significant latency and computational overhead, as they may consume substantial token budgets before reaching the optimal LLM. This makes them impractical for high-volume online routing scenarios with tight token budgets and low-latency requirements.

If no useful routing information can be inferred from historical data, no matter whether it is related to model capability or query features, then the system would need to query all LLMs to make informed routing decisions. This is because in the worst case, under an adversary query targeting the LLM query order in the system, all LLMs may present extremely low performance, except the last LLM. In this situation, no strategy can be adopted to improve overall routing performance without assessing all LLMs. Therefore, it is important and natural to establish our assumption or use a related notion, e.g., Assumption 1, that links historical information with future queries to obtain meaningful routing information and improve routing performance.

B.2 Proof of Lemma 2

Lemma 2. Suppose that $\forall j \in Q$, and $\forall j' \in R_j$, we have $\|\text{EMB}(j) - \text{EMB}(j')\|_2 \leq \eta$, and that Assumption 1 holds. Then, the offline approximate optimum $\hat{C}_{opt}$ satisfies $\left| \frac{\hat{C}_{opt} - C_{opt}}{C_{opt}} \right| \leq O(\delta)$.

Proof. Let x_1 denote the optimal solution to C_{opt} , and x_2 the optimal solution to $\hat{C}_{opt}$.

Case 1: $C_{opt} \geq \hat{C}_{opt}$. We have

$$\begin{aligned} \left| \frac{\hat{C}_{opt} - C_{opt}}{C_{opt}} \right| &= \left| \frac{\sum_j \sum_i \hat{d}_{ij} x_{2ij} - \sum_j \sum_i d_{ij} x_{1ij}}{\sum_j \sum_i d_{ij} x_{1ij}} \right| \\ &\leq \left| \frac{\sum_j \sum_i (\hat{d}_{ij} - d_{ij}) x_{1ij}}{\sum_j \sum_i d_{ij} x_{1ij}} \right| \\ &= \left| \frac{\sum_j \sum_i \left(\frac{1}{|R_j|} \sum_{q \in R_j} d_{iq} - d_{ij} \right) x_{1ij}}{\sum_j \sum_i d_{ij} x_{1ij}} \right| \\ &\leq \left| O(\delta) \frac{\sum_j \sum_i d_{ij} x_{1ij}}{\sum_j \sum_i d_{ij} x_{1ij}} \right| \\ &\leq O(\delta) \end{aligned}$$

where the second inequality follows from the fact that $\sum_j \sum_i \hat{d}_{ij} x_{1ij} \leq \sum_j \sum_i \hat{d}_{ij} x_{2ij}$ when x_{2ij} is the optimal solution to $\hat{C}_{opt}$, and the fourth inequality follows Assumption 1.

Case 2: $C_{opt} < \hat{C}_{opt}$. We have

$$\begin{aligned} \left| \frac{\hat{C}_{opt} - C_{opt}}{C_{opt}} \right| &= \left| \frac{\sum_j \sum_i \hat{d}_{ij} x_{2ij} - \sum_j \sum_i d_{ij} x_{1ij}}{\sum_j \sum_i d_{ij} x_{1ij}} \right| \\ &\leq \left| \frac{\sum_j \sum_i (\hat{d}_{ij} - d_{ij}) x_{2ij}}{\sum_j \sum_i d_{ij} x_{1ij}} \right| \\ &= \left| \frac{\sum_j \sum_i \left(\frac{1}{|R_j|} \sum_{q \in R_j} d_{iq} - d_{ij} \right) x_{2ij}}{\sum_j \sum_i d_{ij} x_{1ij}} \right| \\ &\leq \left| O(\delta) \frac{\sum_j \sum_i d_{ij} x_{2ij}}{\sum_j \sum_i d_{ij} x_{1ij}} \right| \\ &\leq O(\delta) \end{aligned}$$

where the second inequality uses the fact that $\sum_j \sum_i d_{ij} x_{2ij} \leq \sum_j \sum_i d_{ij} x_{1ij}$ when x_{1ij} is the optimal solution to C_{opt} , and the fourth inequality follows Assumption 1 and the final inequality follows $\sum_j \sum_i d_{ij} x_{2ij} \leq \sum_j \sum_i d_{ij} x_{1ij}$. $\square$

B.3 Proof of Lemma 3

Lemma 3. Let Φ be an ϵ -net and assume that

$$\frac{C_{opt}}{s_{max}} \geq \Omega\left(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)}\right).$$

If $\forall i$, $|\hat{C}_i(P) - \epsilon \hat{C}_i| \leq z_i$, then

$$\sum_i z_i \leq O\left(\epsilon^2 \sqrt{(1+\delta) C_{opt} \hat{C} / \alpha}\right).$$

Proof. Inspired by the techniques used in [11, 39, 19], we first prove that, under the given setting, the partial performance score can be accurately estimated with high probability. Specifically, for each $i \in [M]$ and $\gamma \in \Phi$, we evaluate the following probability

$$\Pr(|\hat{C}_i(P) - \epsilon \hat{C}_i| > z_i).$$

Observe that, for any $j \in P$, the variance satisfies $\text{VAR}[\alpha \hat{d}_{ij} x_{ij}(\gamma^*)] \leq \frac{1}{|Q|} \|\hat{C}_i\|_2^2$, which implies $\sum_{j \in P} \text{VAR}[\alpha \hat{d}_{ij} x_{ij}(\gamma^*)] \leq \epsilon \|\hat{C}_i\|_2^2$. By applying Bernstein's inequality, we obtain

$$\Pr(|\hat{C}_i(P) - \epsilon \hat{C}_i| > z_i) \leq 2 \exp \left(-\frac{z_i^2/2}{\epsilon \|\hat{C}_i\|_2^2 + \frac{z_i}{3} \alpha s_{max}} \right).$$

We aim to find such z_i that this probability is below a given tolerance τ . Set RHS as τ and solve for z_i , we obtain

$$z_i = \frac{\frac{2}{3} \log \frac{2}{\tau} \alpha s_{max} + \sqrt{(\frac{2}{3} \log \frac{2}{\tau} \alpha s_{max} + 8\epsilon \log \frac{2}{\tau} \|\hat{C}_i\|_2^2)}}{2} \leq \frac{2}{3} \log \frac{2}{\tau} \alpha s_{max} + \sqrt{2\epsilon \log \frac{2}{\tau} \|\hat{C}_i\|_2^2}$$

Without affecting the analysis, we define $z_i = \frac{2}{3} \log \frac{2}{\tau} \alpha s_{max} + \sqrt{2\epsilon \log \frac{2}{\tau} \|\hat{C}_i\|_2^2}$ for simplicity.

Set $\tau = \epsilon/(M|\Phi|)$, and use $\frac{C_{opt}}{s_{max}} \geq \Omega(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)})$, we have

$$\begin{aligned} \sum_i z_i &= \sum_i \frac{2}{3} \log \frac{2}{\tau} \alpha s_{max} + \sum_i \sqrt{2\epsilon \log \frac{2}{\tau} \|\hat{C}_i\|_2^2} \\ &\leq O(M \log \frac{1}{\tau} \alpha s_{max}) + \sum_i \sqrt{2\epsilon \log \frac{2}{\tau} \hat{C}_i s_{max}} \\ &\leq O(\epsilon^3 C_{opt}(1+\delta)) + \sqrt{2M\epsilon \log \frac{2}{\tau} \hat{C} s_{max}} \\ &\leq O(\epsilon^3 C_{opt}(1+\delta)) + O(\epsilon^2 \sqrt{(1+\delta)}) \sqrt{C_{opt} \hat{C} / \alpha} \\ &\leq O\left(\epsilon^2 \sqrt{(1+\delta) C_{opt} \hat{C} / \alpha}\right) \end{aligned}$$

This completes the proof. $\square$

B.4 Proof of Lemma 4

Lemma 4. *If $\forall i$, it holds that $|\sum_{j \in P} \hat{g}_{ij} x_{ij}(\gamma^*) - \epsilon \sum_j \hat{g}_{ij} x_{ij}(\gamma^*)| \leq O(z_i)$, then there exists a control parameter $\alpha > 0$, such that $\forall i$, (1) $|\hat{C}_i(P) - \epsilon \hat{C}_i| \leq z_i$, and (2) $|\hat{C}_i(P) - \epsilon E_i| \leq O(z_i)$.*

Proof. Define $M_i := \sum_{j \in P} \hat{d}_{ij} x_{ij}(\gamma^*) - \epsilon \sum_j \hat{d}_{ij} x_{ij}(\gamma^*)$ and $H_i := \sum_{j \in P} \hat{d}_{ij} x_{ij}(\gamma^*) - \epsilon \sum_j \hat{d}_{ij} x_{ij}(\gamma^*)$. Then, we aim to ensure the following two conditions hold: (1) $\alpha |M_i| \leq z_i$, and (2) $\alpha |H_i| \leq O(z_i)$.

If $M_i = 0$, condition (1) holds trivially for any $\alpha > 0$; similarly, if $H_i = 0$, condition (2) also holds for any $\alpha > 0$.

Now consider the case where $M_i \neq 0$, then it requires that $\alpha \leq \frac{z_i}{|M_i|}$, and when $H_i \neq 0$, we need to have $\alpha \leq \frac{O(z_i)}{|H_i|}$. Let I be the set of indice i for which $H_i \neq 0$ or $M_i \neq 0$. For each $i \in I$, we have

$$V_{i,1} = \begin{cases} \frac{z_i}{|M_i|} & \text{if } M_i \neq 0 \\ \infty & \text{if } M_i = 0 \end{cases} \quad V_{i,2} = \begin{cases} \frac{O(z_i)}{|H_i|} & \text{if } H_i \neq 0 \\ \infty & \text{if } H_i = 0 \end{cases}$$

Let $V_i := \min\{V_{i,1}, V_{i,2}\}$, and set

$$\alpha = \inf_{i \in I} \{V_i\}$$

Since $V_i > 0$ for all $i \in I$, it follows that $\alpha > 0$. Therefore, we find a control parameter $\alpha > 0$ such that both conditions (1) and (2) are satisfied. $\square$

B.5 Proof of Lemma 5

Lemma 5. Let Φ be an ϵ -net. If $\frac{C_{opt}}{s_{max}} \geq \Omega(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)})$, then $C_{est}(Y) \geq (1 - O(\epsilon))C_{est}$.

Proof. We first prove that $\forall i$, it holds that $\max\{\hat{C}_i, F_i(\gamma^*)\} - C_{est,i} \leq \frac{O(z_i)}{\epsilon}$ where

$$F_i(\gamma^*) := \gamma_i^* B_i + \sum_j (\alpha \hat{d}_{ij} - \gamma_i^* \hat{g}_{ij}) x_{ij}(\gamma^*)$$

From Equation (5), it follows immediately that $\sum_i F_i(\gamma^*) = F(\gamma^*)$.

Case 1: $\gamma_i^* > 0$. In this case,

$$\max\{\hat{C}_i, F_i(\gamma^*)\} - C_{est,i} = \max\{\hat{C}_i, \hat{C}_i + \gamma_i^* (B_i - \sum_j \hat{g}_{ij} x_{ij}(\gamma^*))\} - \min\{E_i, \hat{C}_i\}$$

which follows $C_{est,i} = \min\{E_i, \hat{C}_i\}$. We consider two sub-cases separately:

Sub-case 1.1: $\sum_j \hat{g}_{ij} x_{ij}(\gamma^*) \leq B_i$.

Then,

$$\max\{\hat{C}_i, F_i(\gamma^*)\} - C_{est,i} = \gamma_i^* (B_i - \sum_j \hat{g}_{ij} x_{ij}(\gamma^*)) \leq B_i - \sum_j \hat{g}_{ij} x_{ij}(\gamma^*)$$

Due to complementary slackness conditions over observed queries P , we have $\sum_{j \in P} \hat{g}_{ij} x_{ij}(\gamma^*) = \epsilon B_i$. Thus, it follows that

$$\begin{aligned} & \left| \sum_{j \in P} \hat{g}_{ij} x_{ij}(\gamma^*) - \epsilon \hat{g}_{ij} x_{ij}(\gamma^*) \right| = \epsilon |B_i - \hat{g}_{ij} x_{ij}(\gamma^*)| \leq O(z_i) \\ \Rightarrow \max\{\hat{C}_i, F_i(\gamma^*)\} - C_{est,i} & \leq \frac{O(z_i)}{\epsilon} \end{aligned}$$

Sub-case 1.2: $\sum_j \hat{g}_{ij} x_{ij}(\gamma^*) > B_i$.

We obtain,

$$\max\{\hat{C}_i, F_i(\gamma^*)\} - C_{est,i} = \hat{C}_i - E_i$$

From Lemma 4, it can be expanded and bounded as:

$$\begin{aligned} \hat{C}_i - E_i &= \frac{1}{\epsilon} (\sum_j \epsilon \alpha \hat{d}_{ij} x_{ij}(\gamma^*) - \epsilon E_i) \leq \frac{O(z_i)}{\epsilon} \\ \Rightarrow \max\{\hat{C}_i, F_i(\gamma^*)\} - C_{est,i} &\leq \frac{O(z_i)}{\epsilon} \end{aligned}$$

Case 2: $\gamma_i^* = 0$. Then,

$$\max\{\hat{C}_i, F_i(\gamma^*)\} - C_{est,i} = \hat{C}_i - \min\{\hat{C}_i, E_i\}$$

When $\sum_j \hat{g}_{ij} x_{ij}(\gamma^*) \leq B_i$, this difference $\hat{C}_i - \min\{\hat{C}_i, E_i\} = 0$. Thus, we only consider the situation $\sum_j \hat{g}_{ij} x_{ij}(\gamma^*) > B_i$, leading to

$$\max\{\hat{C}_i, F_i(\gamma^*)\} - C_{est,i} = \hat{C}_i - \min\{\hat{C}_i, E_i\} \leq \hat{C}_i - E_i \leq \frac{O(z_i)}{\epsilon}$$

which follows Lemma 4.

Lower Bound on $C_{est}(Y)$. From Lemma 3 and Lemma 4, we have,

$$\begin{aligned} C_{est,i}(Y) &= \min\{\hat{C}_i - \hat{C}_i(P), E_i - \sum_{j \in P} \alpha \hat{d}_{ij} x_{ij}(\gamma^*)\} \\ &\geq \min\{(1 - \epsilon)\hat{C}_i - z_i, (1 - \epsilon)E_i - O(z_i)\} \\ &\geq (1 - \epsilon)C_{est,i} - O(z_i) \end{aligned}$$

where second inequality follows that $|\hat{C}_i(Y) - (1 - \epsilon)\hat{C}_i| \leq z_i$ and $|\hat{C}_i - E_i| \leq \frac{O(z_i)}{\epsilon}$.

Summing over all i , we get $C_{est}(Y) \geq (1 - \epsilon)C_{est} - \sum_i O(z_i)$. Due to the fact that $\sum_i z_i \leq O\left(\epsilon^2 \sqrt{(1 + \delta)C_{opt}\hat{C}/\alpha}\right)$, we finally obtain

$$\begin{aligned} C_{est}(Y) &\geq (1 - \epsilon)C_{est} - O\left(\epsilon^2 \sqrt{(1 + \delta)C_{opt}\hat{C}/\alpha}\right) \\ \Rightarrow C_{est}(Y) &\geq (1 - O(\epsilon))C_{est} \end{aligned}$$

This completes the proof. $\square$

B.6 Proof of Theorem 1

Theorem 1. For any given query set Q with random arrival order, Algorithm 1 satisfies $\frac{C_{alg}}{C_{opt}} \geq 1 - O(\epsilon + \delta)$ assuming $\frac{C_{opt}}{s_{max}} \geq \Omega\left(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)}\right)$, where s_{max} is the maximum performance score obtained for any query, and Φ is an ϵ -net defined over all possible routing strategies $x(\gamma)$.

Proof. **Case** $\gamma^* \in \Phi$. According to Lemma 3 and 5, we have a union bound over Φ . This union bound implies that with probability greater than $1 - \epsilon$, $\sum_i z_i \leq O(\epsilon^2 \sqrt{(1 + \delta)C_{opt}\hat{C}})$ holds. Therefore, we have that

$$\max\{\hat{C}, F(\gamma^*)\} - C_{est} \leq \frac{1}{\epsilon} \sum_i z_i \Rightarrow \max\{\hat{C}, F(\gamma^*)\} - C_{est} \leq O\left(\epsilon \sqrt{(1 + \delta)C_{opt}\hat{C}/\alpha}\right)$$

- When $(1 + \delta)C_{opt} \geq \hat{C}/\alpha$, we have $\max\{\hat{C}, F(\gamma^*)\} - C_{est} \leq O(\epsilon(1 + \delta)C_{opt})$.

By weak duality, we have $C_{est} \leq \alpha\hat{C}_{opt} \leq F(\gamma^*)$. Also, we have $\frac{\hat{C}_{opt}}{C_{opt}} \in [1 - O(\delta), 1 + O(\delta)]$ (from Lemma 2). Therefore, we obtain that $\alpha(1 - O(\delta))C_{opt} - C_{est} \leq O(\epsilon(1 + \delta)C_{opt}) \Rightarrow C_{est}/\alpha \geq (1 - O(\epsilon + \delta))C_{opt}$.

- When $(1 + \delta)C_{opt} < \hat{C}/\alpha$, by the same weak duality, we have $\hat{C} - \alpha(1 + O(\delta))C_{opt} \leq O(\epsilon\hat{C}/\alpha)$, which further implies that $\hat{C} \leq \alpha \frac{1+O(\delta)}{1-O(\epsilon)/\alpha} C_{opt}$. Therefore, we have $\alpha(1 + \delta)C_{opt} - C_{est} \leq O(\epsilon\hat{C}) \Rightarrow C_{est}/\alpha \geq (1 - O(\epsilon + \delta))C_{opt}$.

We can define $C_{alg} := \sum_i \min\{T_i, \sum_j d_{ij}x_{ij}(\gamma^*)\}$, where $T_i := \sum_j^t d_{ij}x_{ij}(\gamma^*)$ with $t = \arg\max_t \sum_j^t g_{ij}x_{ij}(\gamma^*)$, s.t. $\sum_j^t g_{ij}x_{ij}(\gamma^*) \leq B_i$.

Furthermore, we have $C_{est}/\alpha = \sum_i \min\{E_i/\alpha, \sum_j \hat{d}_{ij}x_{ij}(\gamma^*)\}$, where $E_i/\alpha = \sum_j^k \hat{d}_{ij}x_{ij}(\gamma^*)$ with $k = \arg\max_k \sum_j^k \hat{g}_{ij}x_{ij}(\gamma^*)$, s.t. $\sum_j^k \hat{g}_{ij}x_{ij}(\gamma^*) \leq B_i$.

Let t_i^* be the largest t such that $\sum_j^{t_i^*} g_{ij}x_{ij}(\gamma^*) \leq B_i$. According to Assumption 1, for each $i \in [M]$, we have

$$(1 - O(\delta)) \sum_j^{t_i^*} \hat{g}_{ij}x_{ij}(\gamma^*) \leq \sum_j^{t_i^*} g_{ij}x_{ij}(\gamma^*) \leq (1 + O(\delta)) \sum_j^{t_i^*} \hat{g}_{ij}x_{ij}(\gamma^*)$$

Therefore, we have

$$\sum_j^{t_i^*} \hat{g}_{ij}x_{ij}(\gamma^*) \leq B_i/(1 - O(\delta)) = (1 + O(\delta))B_i \quad (7)$$

Let k_i^* be the largest k such that $\sum_j^{k_i^*} \hat{g}_{ij}x_{ij}(\gamma^*) \leq B_i$. Without loss of generalizability, assume $t_i^* \geq k_i^*$. Then, there exists a set of tail queries, A_i , which makes Equation (7) exceed B_i by at most

$O(\delta)B_i$. Then, we define excess performance contributed by these tail queries as $\sum_{j \in A_i} \hat{d}_{ij}x_{ij}(\gamma^*)$. Because the query order is random, we consider the expected excess performance

$$\mathbb{E}\left[\sum_{j \in A_i} \hat{d}_{ij}x_{ij}(\gamma^*)\right] = \frac{O(\delta)B_i}{B_i}E_i/\alpha = O(\delta)E_i/\alpha$$

and the variance satisfies $\text{VAR}[\sum_{j \in A_i} \hat{d}_{ij}x_{ij}(\gamma^*)] \leq O(\delta)\|E_i/\alpha\|_2^2$. Using Bernstein's inequality, we have

$$\Pr\left(\left|\sum_{j \in A_i} \hat{d}_{ij}x_{ij}(\gamma^*) - O(\delta)E_i/\alpha\right| > e_i\right) \leq 2 \exp\left(-\frac{e_i^2/2}{O(\delta)\|E_i/\alpha\|_2^2 + \frac{e_i}{3}s_{max}}\right)$$

Follow Lemma 3, set RHS as $\epsilon/(M|\Phi|)$, use $\frac{C_{opt}}{s_{max}} \geq \Omega(\frac{\alpha M \log(M|\Phi|/\epsilon)}{\epsilon^3(1+\delta)})$, and union bound for all γ^* and i , then with probability greater than $1 - \epsilon$,

$$\sum_i e_i \leq O\left(\sqrt{\epsilon^3(1+\delta)O(\delta)C_{opt}E/\alpha^2}\right)$$

where we denote $E = \sum_i E_i$.

Based on this, we have

$$\begin{aligned} \left|\sum_i T_i - \sum_i E_i/\alpha\right| &\leq \left|\sum_i \sum_j^{t_i^*} d_{ij}x_{ij}(\gamma^*) - \sum_i \sum_j^{k_i^*} \hat{d}_{ij}x_{ij}(\gamma^*)\right| \\ &\leq \left|(1+O(\delta))\sum_i \sum_j^{t_i^*} \hat{d}_{ij}x_{ij}(\gamma^*) - \sum_i \sum_j^{k_i^*} \hat{d}_{ij}x_{ij}(\gamma^*)\right| \\ &\leq \left|O(\delta)\sum_i E_i/\alpha + (1+O(\delta))\sum_{j \in A_i} \sum_i \hat{d}_{ij}x_{ij}(\gamma^*)\right| \\ &\leq \left|O(\delta)\sum_i E_i/\alpha + (1+O(\delta))(O(\delta)E/\alpha + \sum_i e_i)\right| \\ &\leq \left(O(\delta) + O\left(\sqrt{\epsilon^3(1+\delta)O(\delta)C_{opt}/E}\right)\right)\sum_i E_i/\alpha \\ &\leq \left(O(\delta) + O\left(\sqrt{\epsilon^3(1+\delta)O(\delta)/(1-O(\epsilon+\delta)\alpha)}\right)\right)\sum_i E_i/\alpha \\ &\leq \left(O(\delta) + O\left(\epsilon^{3/2}\sqrt{\delta/\alpha}\right)\right)\sum_i E_i/\alpha \end{aligned}$$

where the penultimate inequality follows $E/\alpha \geq C_{est}/\alpha \geq (1-O(\epsilon+\delta))C_{opt}$.

Finally, it leads to $|C_{alg} - C_{est}/\alpha| \leq \left(O(\delta) + O\left(\epsilon^{3/2}\sqrt{\delta/\alpha}\right)\right)C_{est}/\alpha$. Because $C_{est}/\alpha \geq (1-O(\epsilon+\delta))C_{opt}$, we obtain that $\frac{C_{alg}}{C_{opt}} \geq 1 - O(\epsilon+\delta)$.

Case $\gamma^* \notin \Phi$. As Φ is a ϵ -net, there exists an γ' such that $\forall i, j, |x_{ij}(\gamma^*) - x_{ij}(\gamma')| \leq \epsilon$. Therefore, we obtain

$$\begin{aligned} |\hat{C}_i(\gamma^*, P) - \epsilon \hat{C}_i(\gamma^*)| &\leq |\hat{C}_i(\gamma', P) - \epsilon \hat{C}_i(\gamma')| + |\hat{C}_i(\gamma', P) - \hat{C}_i(\gamma^*, P)| + |\epsilon \hat{C}_i(\gamma') - \epsilon \hat{C}_i(\gamma^*)| \\ &\leq z'_i + \epsilon \hat{C}_i(\gamma^*, P) + \epsilon^2 \hat{C}_i(\gamma^*) \\ &\leq z'_i + O(\epsilon^2 \hat{C}_i(\gamma^*)) \end{aligned}$$

Similarly, for each i , we have

$$\begin{aligned}
\left| \sum_{j \in A_i} \hat{d}_{ij} x_{ij}(\gamma^*) - O(\delta) E_i(\gamma^*) / \alpha \right| &\leq \left| \sum_{j \in A_i} \hat{d}_{ij} x_{ij}(\gamma') - O(\delta) E_i(\gamma') / \alpha \right| \\
&\quad + \left| \sum_{j \in A_i} \hat{d}_{ij} x_{ij}(\gamma') - \sum_{j \in A_i} \hat{d}_{ij} x_{ij}(\gamma^*) \right| \\
&\quad + \left| O(\delta) E_i(\gamma') / \alpha - O(\delta) E_i(\gamma^*) / \alpha \right| \\
&\leq e'_i + \epsilon \sum_{j \in A_i} \hat{d}_{ij} x_{ij}(\gamma^*) + \epsilon O(\delta) E_i(\gamma^*) / \alpha \\
&\leq e'_i + O(\epsilon \delta E_i(\gamma^*) / \alpha)
\end{aligned}$$

where, with slight abuse of notation, we use $E_i(\gamma^*)$ to denote the value computed under γ^* and $E_i(\gamma')$ under γ' .

By summing over all i , and following Lemma 3 as well as the proof in case $\gamma^* \in \Phi$, we obtain $C_{est}/\alpha \geq (1 - O(\epsilon + \delta))C_{opt}$, and $|C_{alg} - C_{est}/\alpha| \leq \left(O(\delta) + O\left(\epsilon^{3/2} \sqrt{\delta/\alpha}\right) \right) C_{est}/\alpha$.

Therefore, Algorithm 1 satisfies that $\frac{C_{alg}}{C_{opt}} \geq 1 - O(\epsilon + \delta)$. $\square$

C Additional Experimental Results

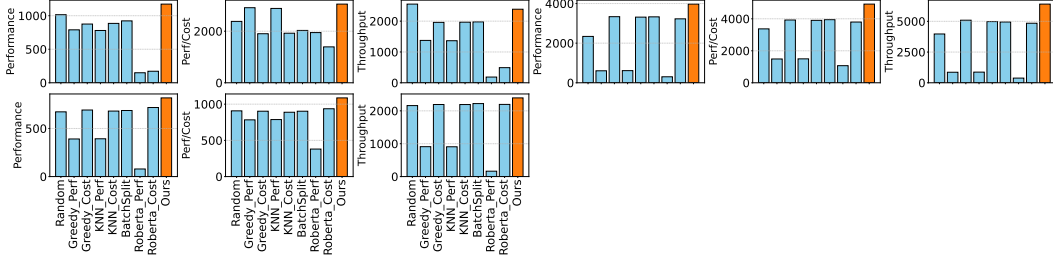


Figure 7: Results under the worst-case query order, where queries are sorted in descending order of cost. (Left to right): the first three subfigures correspond to RouterBench, the next three to SPROUT, and the last three to Open LLM Leaderboard v2.

C.1 Query Arrival Order

We evaluate the robustness of our algorithm under varying query arrival orders. Specifically, we consider two settings: (1) a randomized setting, where test queries are independently shuffled 100 times to reflect realistic, unpredictable online environments; and (2) a worst-case setting, where queries are sorted in descending order of their maximum cost across all models. This adversarial order simulates scenarios in which expensive queries arrive early and are more likely to exhaust the budget. Figure 2 shows that our algorithm consistently outperforms all baselines across metrics and benchmarks under random permutations. Even under the worst-case order (Figure 7), our method maintains the advantage, achieving the best performance across all metrics on SPROUT and Open LLM Leaderboard v2, and outperforming all baselines in terms of performance and cost-efficiency on RouterBench. These results demonstrate the robustness of our algorithm towards varying query orders.

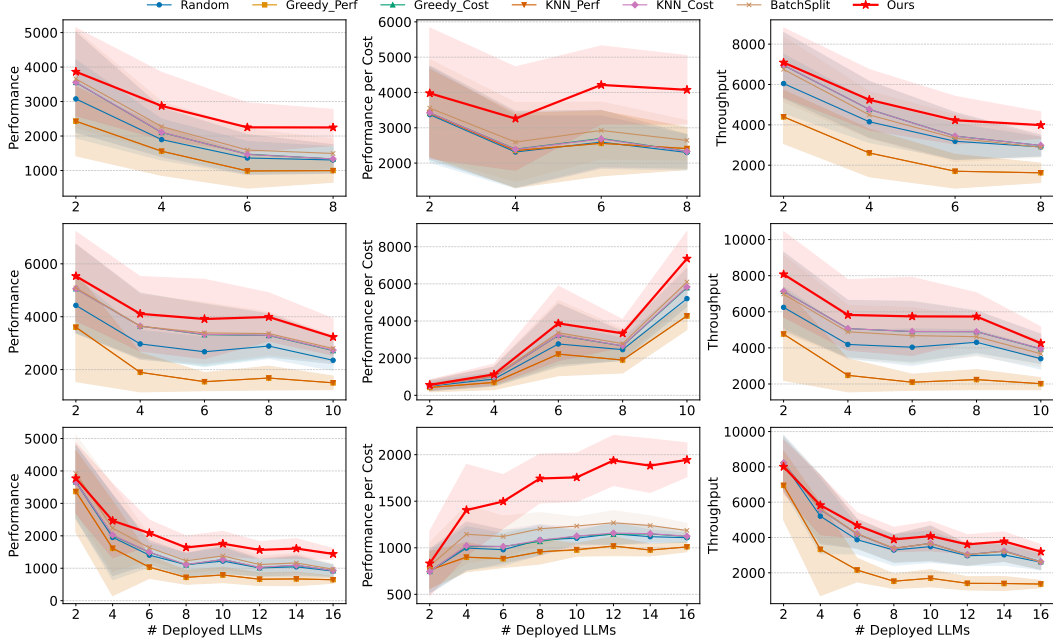


Figure 8: Results when varying the configurations of deployed LLMs. Rows correspond to different datasets: RouterBench (top), SPROUT (middle), and Open LLM Leaderboard v2 (bottom).

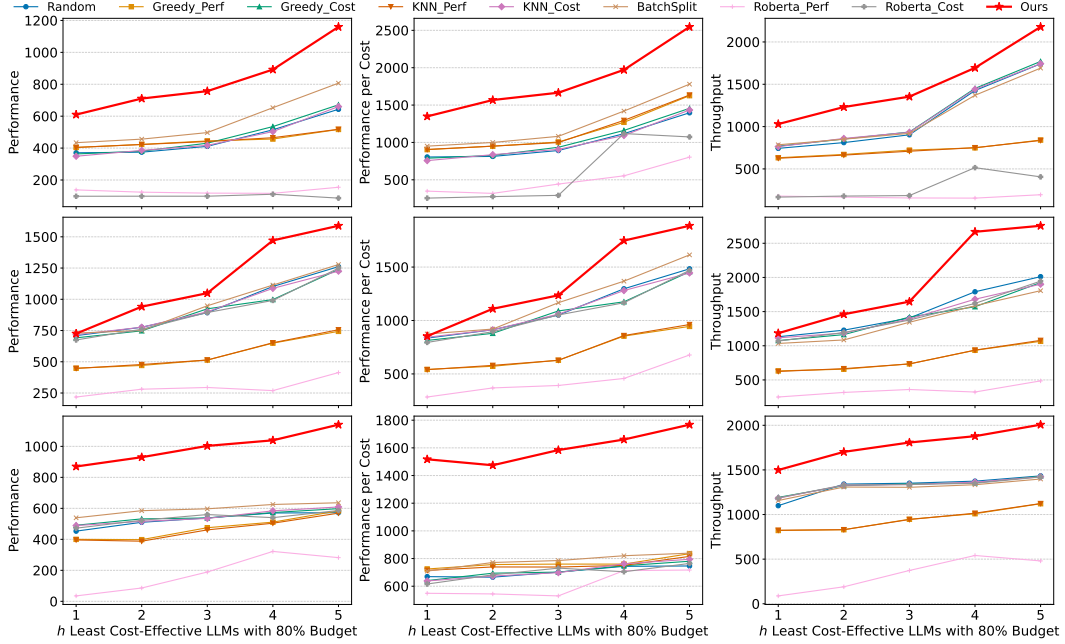


Figure 9: Results under the extreme budget split. Rows correspond to different datasets: RouterBench (top), SPROUT (middle), and Open LLM Leaderboard v2 (bottom).

C.2 Scalability to LLM Deployments.

One of the key advantages of our algorithm is its scalability to varying configurations of LLM deployments. To evaluate the robustness of our algorithm, we vary the configurations of deployed LLMs on each benchmark. For RouterBench, which includes 11 distinct LLMs in total, we vary the

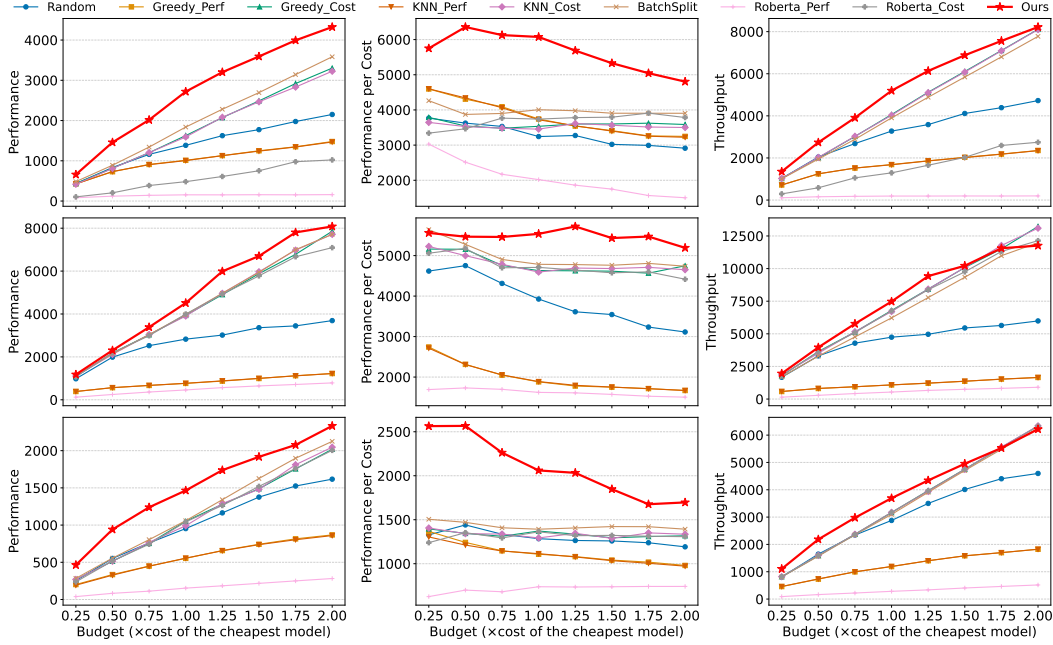


Figure 10: Results when varying total budget B from 0.25 to $2\times$ the cost of the cheapest model. Rows correspond to different datasets: RouterBench (top), SPROUT (middle), and Open LLM Leaderboard v2 (bottom).

deployed LLMs from 2 to 8. For SPROUT, which has 13 different LLMs, we vary the deployed LLMs from 2 to 10. For Open LLM Leaderboard v2, which includes 18 distinct models, we vary the number of deployed LLMs from 2 to 16. In all settings where the number of deployed LLMs is fewer than the maximum, we randomly sample the deployed LLMs and repeat the experiment 10 times to ensure diversity and coverage of possible configurations. Note that in this experiment, we compare only against training-free methods, as retraining models used in model-based methods for each of the numerous settings would incur significant computational and deployment overhead. As shown in Figure 8, our algorithm consistently achieves strong performance across all deployment configurations, where the cost-efficiency gap over other baselines steadily increases. These results demonstrate the robustness and adaptability of our method to diverse and dynamic LLM serving environments.

C.3 Budget Split

One key factor affecting the performance of routing algorithms is the choice of budget split strategy. We extend the default cost-efficiency-based split to five alternative strategies. As shown in Figure 4, our algorithm consistently outperforms all baselines across all benchmarks and metrics under four different split strategies: cost-based, performance-based, uniform, and random. Even under the extreme split setting (Figure 9), where a large portion of the budget is concentrated on a few low-efficiency models, our method maintains leading performance. Notably, when 80% of the budget is allocated to a single model ($h = 1$), our algorithm achieves nearly $2\times$ the performance of the strongest baseline (BatchSplit) on the RouterBench. On Open LLM Leaderboard v2, it also demonstrates approximately $2\times$ higher cost efficiency than all baselines. These results underscore the strong robustness and adaptability of our algorithm against a wide range of complex and imbalanced budget allocation schemes.

C.4 Total Budget

To evaluate the performance of our algorithms in different budget settings, we scale the total budget B from 0.25 to $2\times$ the cost of the cheapest model. Results shown in Figure 10 demonstrate that our algorithm achieves competitive or superior performance compared to all baselines across benchmarks

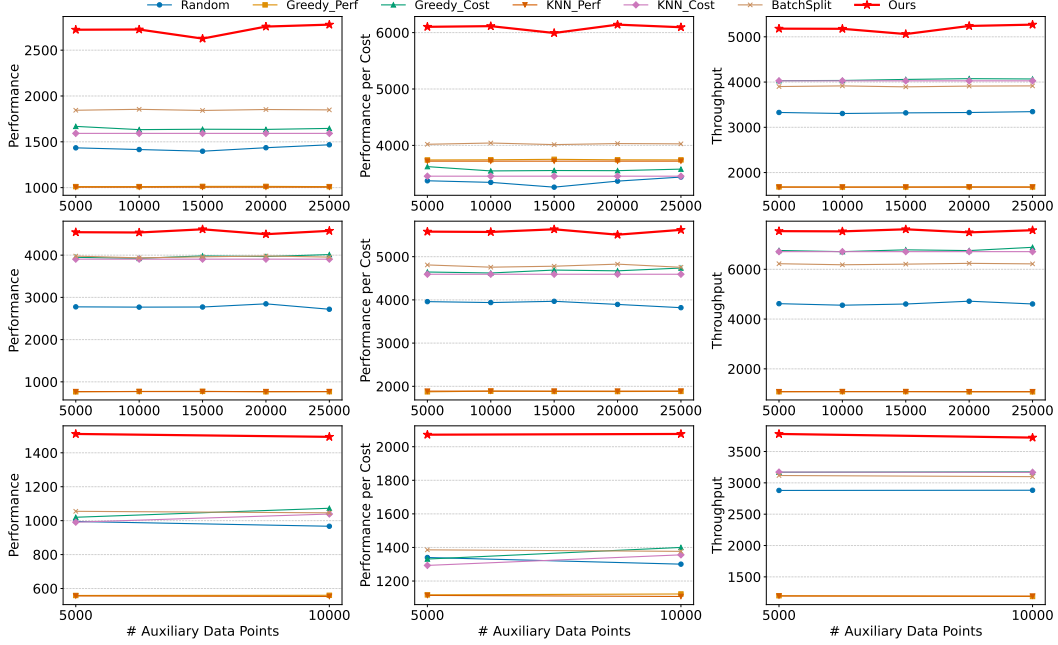


Figure 11: Results when varying the number of historical data points. Rows correspond to different datasets: RouterBench (top), SPROUT (middle), and Open LLM Leaderboard v2 (bottom).

and metrics. Notably, even under the extremely constrained budget, it maintains a clear advantage. For instance, at $B = 0.25$, it achieves nearly $2\times$ the performance and cost efficiency of the strongest baseline (BatchSplit) on the Open LLM Leaderboard v2. These results underscore the robustness of our method and its adaptability to different resource availability scenarios.

Table 7: Per-query routing latency (ms) of different algorithms on RouterBench under varying numbers of queries.

Algorithm	4000 queries	6000 queries	8000 queries	10000 queries
Greedy-Perf	0.203	0.203	0.201	0.203
Greedy-Cost	0.233	0.230	0.226	0.233
KNN-Perf	0.478	0.475	0.469	0.475
KNN-Cost	0.300	0.282	0.281	0.278
BatchSplit	0.300	0.282	0.281	0.278
Ours	0.060	0.060	0.060	0.064

C.5 Routing Overhead & Quality of Historical Data

Routing Overhead. We evaluate the routing latency of Algorithm 1 under varying query volumes. We compare Algorithm 1 with training-free routing baselines, focusing on the core online routing stage that follows the embedding step. This focus is motivated by two key reasons: (1) The embedding step can operate asynchronously and is fully decoupled from the core routing logic, where its outputs are shareable across retrieval, logging, and other downstream tasks. (2) The embedding model is interchangeable (e.g., MiniLM, GTE-small, bge-small), while our core contribution lies in the core routing algorithm itself. As shown in Table 7, our method incurs negligible decision overhead compared to existing training-free baselines. Even under high query volume (10,000 queries), it introduces only 0.064ms of routing latency per query and consistently achieves the lowest decision overhead (except the random routing strategy, which exhibits significantly worse performance).

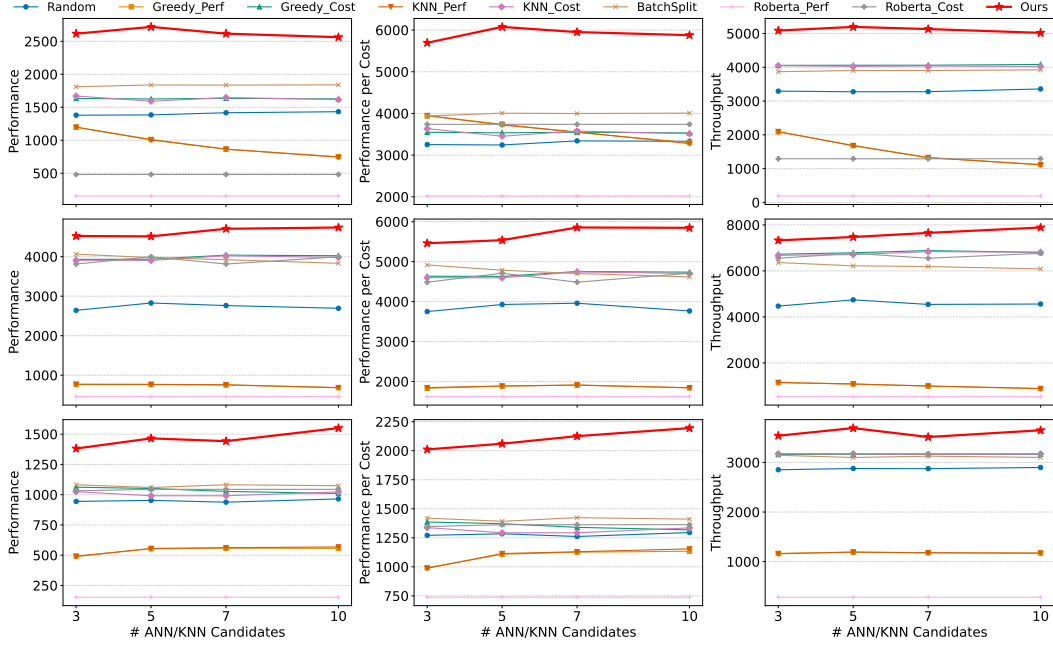


Figure 12: Results when varying the number of search candidates. Rows correspond to different datasets: RouterBench (top), SPROUT (middle), and Open LLM Leaderboard v2 (bottom).

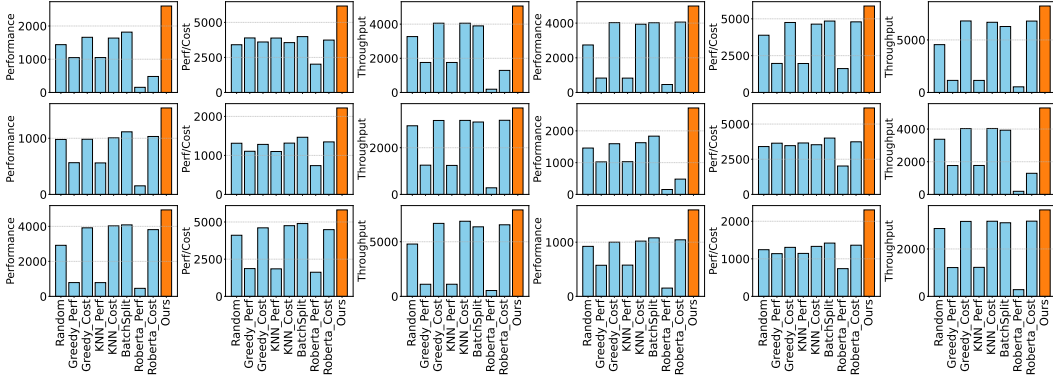


Figure 13: Results using different embedding models. Each group of 9 subfigures corresponds to one embedding: (a) gte-Qwen2-1.5B-instruct (subfigures 1–9), (b) SFR-Embedding-2_R (subfigures 10–18). Within each group, the first three subfigures correspond to RouterBench, the next three to SPROUT, and the last three to Open LLM Leaderboard v2.

Quality of Historical Data. To assess robustness to data quality, we consider a noisy setting with label perturbations and an out-of-distribution (OOD) setting with distribution shift. In the noisy setting, we introduce two types of strong noise simultaneously to the data label. For each performance label d_{ij} , we randomly flip its value with a probability of 20%. For each cost label g_{ij} , we apply multiplicative, mean-preserving noise consisting of: (i) log-normal jitter ($\log \mathcal{N}(0, \sigma^2) - \frac{1}{2}\sigma^2$) with $\sigma = 0.25$, and (ii) a spike factor of 3.0 applied with 2% probability. In the OOD setting, we construct a distribution shift using RouterBench, which contains 13 diverse datasets. Specifically, we use MMLU as the historical dataset and evaluate on the remaining 12 datasets, which differ significantly in data distribution. Results in Table 8 show that our algorithm consistently outperforms all baselines under both noisy label and OOD settings. It achieves the highest overall performance, the best performance-per-cost ratio, and the greatest throughput in both settings, demonstrating strong robustness to label noise and distribution shift.

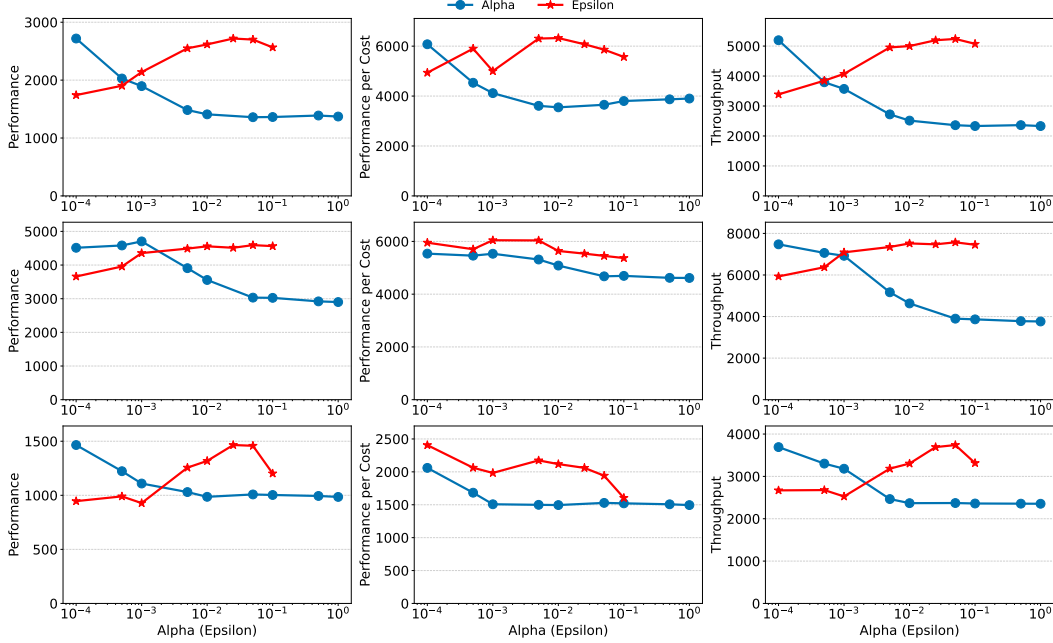


Figure 14: Results when varying α and ϵ . Rows correspond to different datasets: RouterBench (top), SPROUT (middle), and Open LLM Leaderboard v2 (bottom).

Table 8: Routing performance on RouterBench under two challenging historical-data conditions: noisy data label and out-of-distribution (OOD) data. Perf represents the *Performance*, PPC represents *Performance per Cost*, and Tput represents *Throughput*.

Algorithm	Noisy Data Label				Out-of-Distribution (OOD)			
	Perf	Cost	PPC	Tput	Perf	Cost	PPC	Tput
Random	1403.250	0.419	3350.430	3227	1495.35	0.518	2888.14	3308.00
Greedy-Perf	1323.600	0.370	3573.750	2471	807.15	0.252	3204.76	1643.00
Greedy-Cost	1660.200	0.461	3604.720	4047	1677.45	0.561	2989.30	4044.00
KNN-Perf	1323.950	0.369	3585.390	2472	816.10	0.254	3212.31	1658.00
KNN-Cost	1660.200	0.461	3604.720	4047	1679.70	0.561	2993.23	4047.00
BatchSplit	1757.250	0.456	3856.680	3974	1828.55	0.526	3474.42	4283.00
Roberta-Perf	83.000	0.045	1836.330	84	9.50	0.017	559.15	10.00
Roberta-Cost	358.000	0.075	4787.593	1616	1692.65	0.562	3013.92	4037.00
Ours	2319.650	0.440	5221.280	4775	2050.8	0.532	3851.53	4644.00

C.6 Historical Data Size & ANNS/KNN Candidates

Historical Data Size. To evaluate the robustness of our algorithm with respect to the size of historical data, we vary the number of historical data points used in ANNS and KNN from 5000 to 25000. For RouterBench, whose maximum available historical data is 26497, we randomly sample the subset of data points from 5000 to 25000. For RouterBench, which contains up to 26497 historical records, we randomly sample subsets within this range. For SPROUT, with a maximum of 30968 records, we similarly sample subsets from 5000 to 25000. For Open LLM Leaderboard v2, which has 11065 historical records, we sample subsets ranging from 5000 to 10000. As shown in Figure 11, the results clearly show that varying the amount of historical data used in ANNS and KNN has minimal impact on performance, with our algorithm consistently and significantly outperforming all baselines across all settings. This demonstrates the robustness and efficiency of our algorithm: even with as few as 5000 historical records, it remains effective for routing.

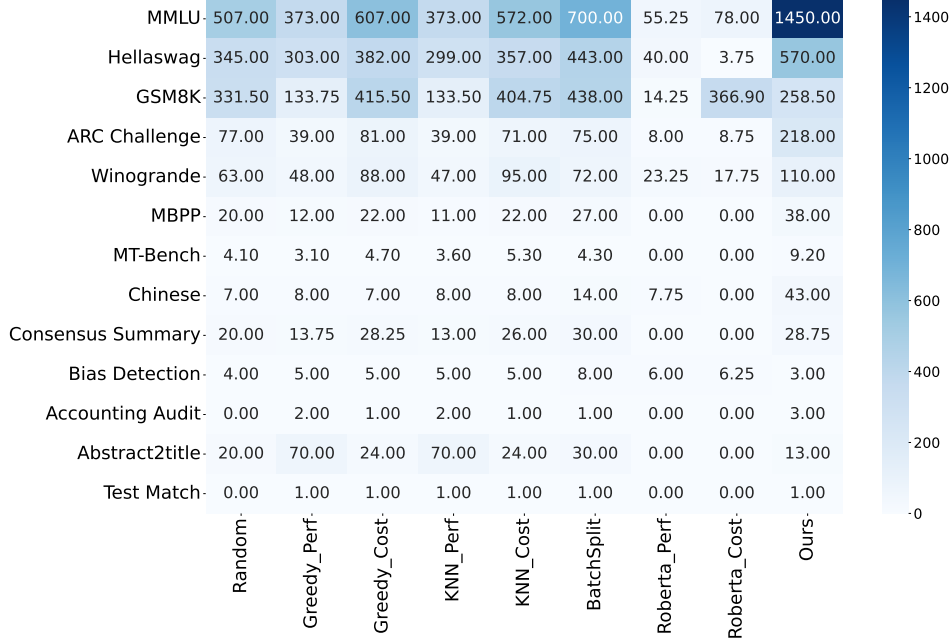


Figure 15: Results across different query types in RouterBench.

Number of ANNS/KNN Candidates. To investigate the impact of the number of search candidates of ANNS and KNN on routing performance, we vary the candidate pool size from the default 5 to 3, 7, and 10. As shown in Figure 12, the results show that performance is marginally affected, with a slight improvement as the candidate pool size increases. Notably, our algorithm consistently maintains a leading position across all settings. Even with as few as 3 search candidates, it outperforms all baselines across benchmarks, demonstrating its stability and robustness to the choice of candidate pool size.

C.7 Detailed Ablation Study

Impact of embedding models. The choice of embedding model is also important for estimating the performance score and cost of incoming queries. To evaluate the adaptivity of our algorithm to different embedding models used in ANNS, we conduct experiments with two larger embedding models: SFR-Embedding-2_R [40] and gte-Qwen2-1.5B-instruct [35]. As shown in Figure 13, our algorithm consistently outperforms all baselines across all benchmarks, demonstrating its robustness to the choice of embedding model.

Impact of α and ϵ . We evaluate the sensitivity of our method to the parameters α and ϵ , as shown in Figure 14. For α , we observe a consistent decline in performance as it increases, with the best result achieved at $\alpha = 0.0001$. This trend aligns with Lemma 4, which shows that a larger α amplifies the gap between the performance of our algorithm on the observed subset P , $\hat{C}(P)$, and its expected value $\epsilon\hat{C}$. This increased discrepancy results in greater deviation in performance on future queries, ultimately leading to a reduction in overall performance on the full query set Q . For the parameter ϵ , we observe that increasing its value initially improves performance, reaching a peak at around $\epsilon = 0.025$, after which further increases lead to a decline. This aligns with the intuition that ϵ controls the number of samples used in the learning stage: too few samples lead to underfitting, while too many may lead to overfitting to the observed data.

C.8 Routing Analysis by Query Types

To gain deeper insights into how our algorithm routes different types of queries, we analyze its routing behavior across the three benchmarks. We categorize query types based on their data sources, as each source is designed to evaluate different LLM capabilities. As shown in Figure 15, 16, and 17, our



Figure 16: Results across different query types in SPROUT.



Figure 17: Results across different query types in Open LLM Leaderboard v2.

algorithm consistently achieves superior or competitive routing performance in most query types across benchmarks. These results demonstrate the generalizability and effectiveness of our algorithm in handling diverse query types, which indicates that its performance is not dependent on a narrow subset of tasks.

D Query-Traffic Considerations in LLM Routing

In the practical LLM-serving systems, an additional important routing factor is query traffic patterns. However, the primary goal of this work is to develop a cost-efficient online routing algorithm, a problem that has gained increasing attention in recent literature [43, 23, 50], especially given the high computational cost of LLMs. To isolate this core algorithmic problem, we adopt a standard assumption widely used in the online algorithms literature [11, 39, 19]: incoming queries arrive in a

randomly permuted order. This assumption, discussed in detail in Appendix B.1, allows us to abstract away complex traffic dynamics and focus on the fundamental cost-efficiency routing problem.

Under this assumption, our algorithm is designed to operate on **query-intrinsic features** that are independent of time and query order, such as response performance score and cost per query across the models. In contrast, **operational-state features**, such as current queue lengths or system load, are beyond the scope of our current formulation and are more aligned with a related but orthogonal problem: load balancing under traffic constraints.

Still, we believe our algorithm offers a natural foundation for incorporating such traffic-aware considerations. One potential approach is to discretize time into short windows and track the per-model backlog of unserved workload. Within each time window, a corresponding "time-bucket" MILP can be solved with additional traffic constraints, and any unserved workload can be carried over to the next window. For instance, let ℓ_{ij} denote the service time of query j on LLM i , b_{it} be the backlog of unserved workload for LLM i at the start of time window t , and A_{it} denote the available service capacity for LLM i during window t . Then, for each window t and LLM i , we can add the following constraint to the MILP:

$$\sum_j \ell_{ij} x_{ij} \leq \max\{0, A_{it} - b_{it}\}, \quad \forall i, t \quad (8)$$

This "time-bucket" formulation helps smooth bursts and reduce queuing delays. We leave the integration of such traffic-aware routing mechanisms to future work.

E Extended Related Work

LLM Routing. Existing LLM routing research primarily falls into two paradigms. The first focuses on improving response quality while managing cost, typically through ensembling outputs from different LLMs [26, 55] or using cascading strategies that query LLMs sequentially based on their capabilities [5, 2, 62, 33]. For instance, LLM-Blender [26] proposes an ensembling framework that combines outputs from multiple open-source LLMs and selects the optimal response. [55] using outputs Wang et al. [55] propose to fuse outputs of expert models that capture complementary aspects of the data distribution, to generate the final answer. Frugal-GPT [5] adopts a sequential querying strategy, invoking LLMs in order of increasing capability until a satisfactory response is obtained. AutoMix [2] first uses a smaller model to self-verify the quality of the response, based on which it dynamically selects a suitably sized model for handling the query. Although these approaches can improve performance, they incur high latency and computational cost due to multiple model invocations per query.

The second paradigm leverages historical or auxiliary datasets to estimate the performance and cost of each query and select the optimal LLM accordingly [43, 50, 23, 13, 37, 27, 48, 52, 15, 20, 56]. In this line of work, several approaches train model-based predictors using these labeled historical datasets. For example, RouterLLM [43] trains a BERT-base or causal LLM on the historical data annotated with human preferences to enhance routing accuracy. CARROT [50] trains Roberta-base models to estimate both performance and cost for each query, enabling selection of the optimal LLM under any user-defined trade-off between quality and cost. HybridLLM [13] adopts the synthetic preference labels from the MixInstruct dataset [26] based on BARTScore [61], and trains a single BERT-based router for query routing. Zooter [37] distills reward signals from training queries and applies tag-based label enhancement to train a routing function that facilitates expert selection. TensorOpera [52] introduces a soft label-based strategy based on the BERT similarity scores to train the BERT-based predictor for query routing. GraphRouter [15] constructs a heterogeneous graph to capture the contextual relationship between the query requirements and the LLM capabilities for informed routing decisions. While these training-based approaches are effective for certain LLM deployment settings, they introduce nontrivial training overhead. Furthermore, adapting them to varying LLM deployment configurations typically requires retraining, which makes them unsuitable for resource-constrained online routing with diverse LLM deployment configurations.

Some methods avoid training by using approaches such as KNN [23] or similarity-weighted (SW) ranking [43] to estimate the performance and cost based on the historical data. While these approaches avoid model training overhead, they still incur substantial computational overhead and latency, as they rely on brute-force comparisons against the entire historical dataset to retrieve similar examples

or compute similarity scores without any optimization. Specifically, they operate with a high search complexity of $O(N)$, where N is the size of the dataset. As a result, these methods are difficult to scale and impractical for high-volume, budget-constrained online routing scenarios.

Recent efforts [47, 42, 44] have explored formulating LLM routing as MILP. For instance, Sakota et al. [47] introduce two MILP-based strategies, targeting performance-oriented and cost-oriented, respectively. However, these methods face significant challenges in high-volume online routing settings, where queries arrive sequentially, rather than simultaneously, making the offline optimal infeasible to compute.

We complement these works by introducing the first training-free and efficient online routing method with provable performance guarantees, tailored for high-volume, budget-constrained, and dynamic LLM-serving environments.

Online Matching. This problem has been extensively studied for decades, with much of the literature focusing on important special cases of the general online submodular welfare maximization problem [29, 32]. These include the classical unweighted online bipartite matching problem [28, 30, 31], its extension to vertex-weighted matching [4, 1], the display ads allocation problem [14, 6], and the AdWords problem [39, 11, 4].

F Broader Impacts

Our algorithm proposes an efficient, training-free solution for online routing in high-volume, budget-constrained LLM serving. It significantly reduces serving costs and improves performance, which enables more sustainable, scalable, and cost-effective multi-LLM deployment. As a technical contribution, it does not pose direct societal risks. We believe it promotes broader accessibility to LLM services, and future work can further address potential concerns by incorporating responsible use guidelines.